



Ministry of higher education
Modern Academy
Computer engineering and information
technology department

SMART PARKING SYSTEM

PREPARED BY

Ammar Abdalaziz Abdalmgid Snosy

Abdelaziz Diab Hassan Mohamed

Abdelrhamn Mohamed Amin Ali

Doaa Mahmoud Fawzy Attya

Ebrahim Qlini Fahmy Qlini

Rana Ehab Naffe Agha

SUPERVISOR

DR.SEHAM MOAWAD

2023

Special Thanks

In the name of Allah, the all merciful,

Praise to Allah for helping us to accomplish this documentation.

Many thanks to my Profs. Of the department for helping us by lectures which were really a good source of information to us.

Special thanks to **Dr. Sabry** and **Dr.sahem** who helps us to arrange our ideas, the steps to begin the project to its end by giving us information and explanation that helps to achieve the project.

Prepared by

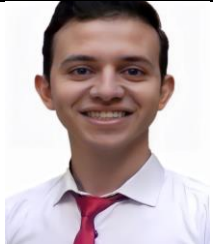
Abdelrhaman Mohamed Amin Ali	
Doaa Mahmoud Fawzy Attya	
Ebrahim Qlini Fahmy Qlini	
Abdelaziz Diab Hassan Mohamed	
Rana Ehab Naffe Agha	
Ammar Abdalaziz Abdalmgid Snosy	

Table of Contents

Chapter (1) Introduction	6
Introduction to Android	7
What is Android?	7
Why Android?	8
Prerequisites of Android	8
Android Architecture	10
Advantages of Android Development	12
Disadvantages of Android	13
Challenges of Android Development	15
Future scope and importance of Android	17
How to kick start with android easily?	17
What is React Native?	18
Advantages of React Native	19
Introduction of Artificial intelligence	20
A Brief History of AI	20
A Brief History of Deep Learning:	20
Introduction to Data	26
Importance of data in our system:	27
Introduction to Embedded System	29
System:	29
Embedded System	29
Structure of embedded systems	29
Basic Structure of an Embedded System	30
Embedded Systems Structure	30
Types of Embedded Systems	31
Stand-alone Embedded Systems	31
Real-time Embedded Systems	31

Network Embedded Systems	32
Mobile Embedded Systems.....	33
Advantages and Disadvantages of Embedded System:	34
Chapter (2) Background.....	35
Abstract	36
Survey	37
Function Requirements:	40
Non-function requirements	43
Database	45
Entity Relationship Diagrams (ERD)	45
Schema	46
Business Model	47
Feasibility study	48
Technical Feasibility	48
Economic feasibility	48
Chapter (3) Design	49
Use case diagram	51
Use case description.....	52
Chapter (4) Deployment & layout.....	76
Mobile App “Rakna”	77
Web App	86
Hardware	90
Ai Face recognition.....	104
Chapter (5) Simulation	107
Simulate parking process in our system	108
First steps to enter parking.....	108
Second steps to exit parking	112
Chapter (6) Conclusion & future works	115

Conclusion	115
Future works	115
Future works for software	115
Future works for hardware	116
References	117
List of Figures.....	118
List of Tables	120

Chapter (1) Introduction

Welcome to the chapter Introduction on Sciences used Our Project! In this chapter, we will explore the essential scientific concepts that form the foundation of our project. By understanding these sciences, we can effectively leverage them to create innovative and impactful solutions.

To begin, we will dive into the Introduction to Android. Android, as a mobile operating system, relies on various scientific principles to enable its functionality. We will explore the underlying concepts of Android architecture, application components, user interface design, and data storage. Understanding these scientific aspects of Android will empower us to develop robust and user-friendly mobile applications.

Next, we will move on to the Introduction to Artificial Intelligence (AI). AI is a branch of computer science that focuses on creating intelligent machines capable of mimicking human intelligence. We will explore the fundamental principles of AI, including machine learning, neural networks, and natural language processing. Understanding the scientific foundations of AI will enable us to harness its power in our project to achieve intelligent and autonomous functionalities.

Following AI, we will delve into the Introduction to Data. Data science plays a crucial role in extracting meaningful insights from vast amounts of information. We will explore the basics of data management, including data collection, storage, and analysis. Additionally, we will touch upon data visualization and statistical analysis techniques that allow us to derive valuable conclusions from data. The scientific understanding of data will enable us to make informed decisions and drive our project's success.

Lastly, we will explore the Introduction to Embedded Systems. Embedded systems are computer systems designed to perform specific tasks within larger systems or devices. We will delve into the scientific principles behind embedded systems, including hardware-software integration, real-time operating systems, and low-power design. Understanding the scientific aspects of embedded

systems will empower us to develop efficient and reliable solutions for our project.

By exploring these scientific disciplines, we aim to leverage their principles and techniques to create a robust and innovative project. Whether you are a student, a professional, or a technology enthusiast, this chapter will provide valuable insights into the scientific foundations that underpin our project's success. Let's embark on this exciting journey of integrating sciences into our project!

Introduction to Android

What is Android?

Google's Android operating system is a mobile-based open-source platform.

Nowadays, most smartphones, tablets, televisions, and other devices like even your

Fitbit run on android. Android is based on top of the Linux kernel (modified version).

Since Android is open-source, anyone can download the source code, edit it according to his requirements, and launch his own custom ROM. As a result, Android is available for almost all devices, irrespective of the manufacturers.

It provides you with facilities like SDK(also known as the software development kit) to write your codes, debug them, and make awesome apps for every generation.

Android Versions

Google first publicly announced Android in November 2007 but was released on 23 SEPTEMBER 2008 to be exact. The first device to bring Android into the market was the HTC Dream with the version Android 1.0. Since then, Google released a lot of android versions such as Apple Pie, Banana Bread, Cupcake, Donut, Éclair, Froyo, Gingerbread,

Jellybeans, Kitkat, Lollipop, marshmallow, Nougat, Oreo, etc. with extra functionalities and new features.

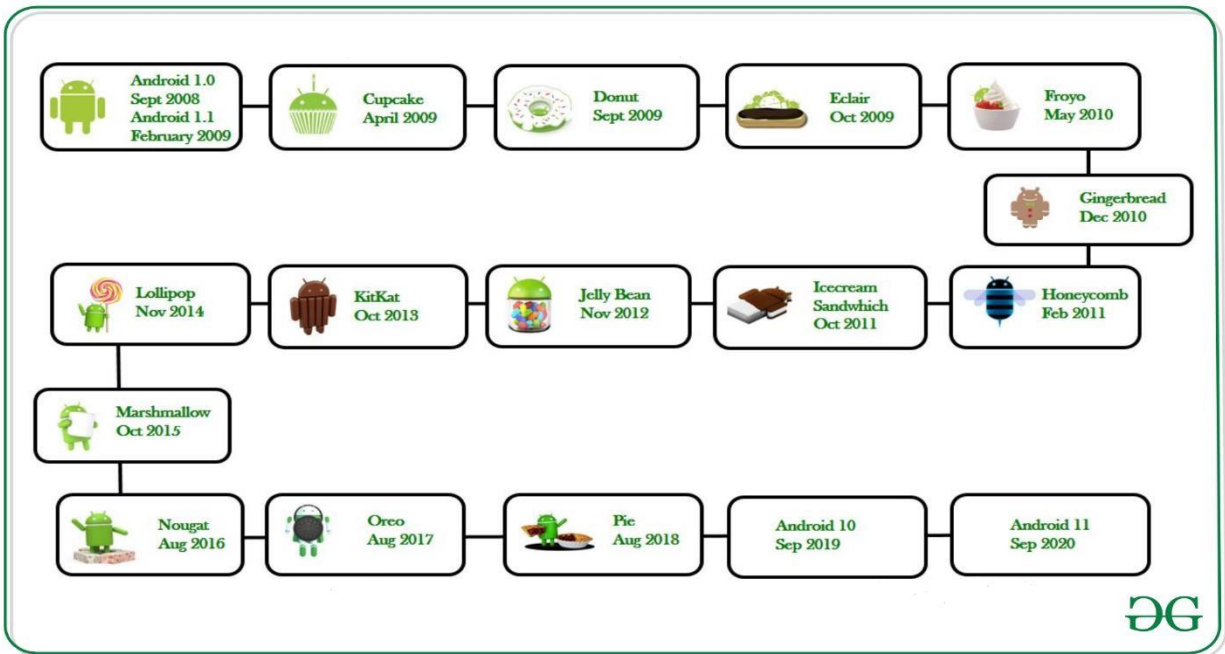


Figure 1: Android types

Why Android?

Android itself being widely used across the world makes it essential for developers to choose android. There are other operating systems such as IOS, Ti zen, etc. Most Apple smartphones use IOS as their mobile operating system. Most of the smartphones available in the market use Android as their operating system, making it available to more users.

It is not just why android is so essential, but several Android features make android quite valuable for the current scenario.

Prerequisites of Android

There are not many prerequisites to start with Android Development. However, before starting with android development, you should make sure that you have gone through the following requirements. The prerequisites can be listed as follows:

1. Java/Kotlin: You should have a basic knowledge of either Java or Kotlin.
2. Database: If you have some knowledge of SQL and SQLite, then it's pretty good.
3. XML: A basic knowledge of XML is required.
4. OOP: Knowing the Object-Oriented Programming(OOP's) concepts would be an add-on and help you during app development.

Features of Android

Android is a powerful open-source operating system that open-source provides immense features and some of these are listed below.

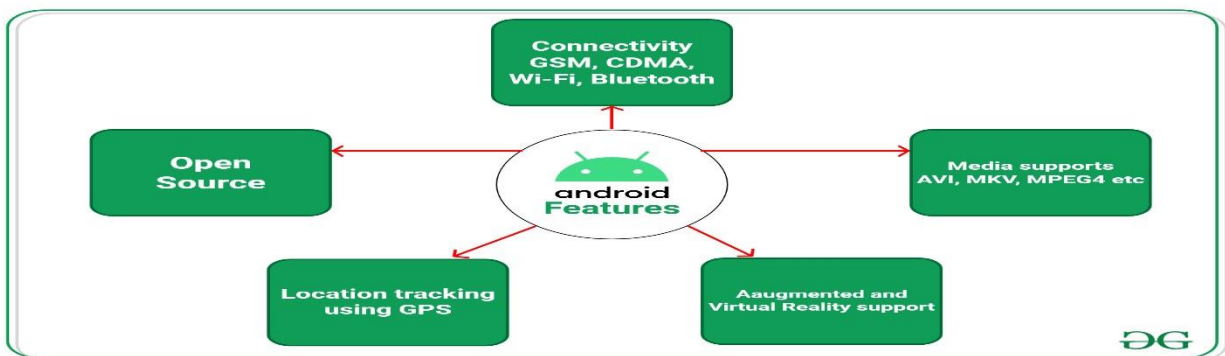


Figure 2: Features of android

- Android Open Source Project so we can customize the OS based on our requirements.
- Android supports different types of connectivity for GSM, CDMA, Wi-Fi, Bluetooth, etc. for telephonic conversation or data transfer.
- Using wi-fi technology we can pair with other devices while playing games or using other applications.
- It contains multiple APIs to support location-tracking services such as GPS.
- We can manage all data storage-related activities by using the file manager.
- It contains a wide range of media supports like AVI, MKV, FLV, MPEG4, etc. to play or record a variety of audio/video.

- It also supports different image formats like JPEG, PNG, GIF, BMP, MP3, etc. □ It supports multimedia hardware control to perform playback or recording using a camera and microphone.
- Android has an integrated open-source WebKit layout-based web browser to support User Interfaces like HTML5, and CSS3.
- Android supports multi-tasking means we can run multiple applications at a time and can switch between them.
- It provides support for virtual reality or 2D/3D Graphics.

Android Architecture

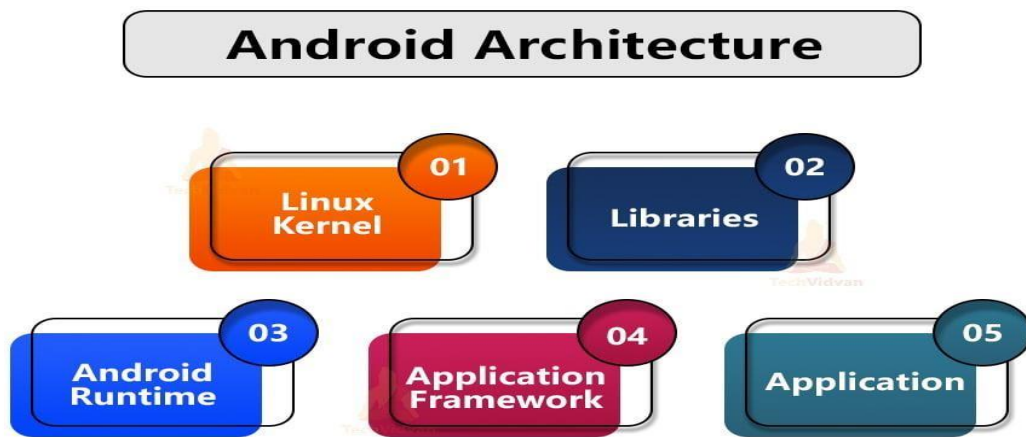


Figure 3: Android Architecture

Any Operating System has its architecture to carry on various functionalities. Similarly, android has its architecture. The android architecture gives us an idea about its design and the build. The architecture can be subdivided into further categories as below: 1. Linux Kernel

Linux Kernel carries all the drivers for the low-level devices like Audio Driver, WiFi Driver, Flash Memory Driver, Bluetooth driver, Camera Driver, Keypad Driver, etc. It is also the abstract layer of android.

2. Libraries

There are several libraries to provide various functionality for purposes like android development. These libraries are written in C/C++ and are an essential part of architecture.

3. Android Runtime

It provides us with an environment for executing and debugging our android applications.

4. Application Framework

It has many packages in it which are implemented in Java.

5. Applications

At the top level of the architecture, you have the applications. Applications can be of system or user or even kept by the OEM manufacturers. Phone, Messaging, Camera, Gallery, etc., are some of the standard applications present in any android device.

Fundamentals of Android development

Regardless of the moving pieces of how applications get converted from source code to running code, from a programming perspective, there are some fundamentals to understand when starting out with Android development.

An Android application consists of one or more of the following four classifications:

- **Activities:** An application that has a visible user interface is implemented via an activity. When you select an application from the Home screen or application launcher, an activity is started.
- **Services:** You can use a service for any application that needs to persist for a long time, such as a network monitor or update-checking application.
- **Content providers:** The easiest way to think about content providers is to view them as a database server. A content provider's job is to manage access to persisted data, such as the contacts on a phone. If your application is very simple, you might not necessarily create a

content provider, however if you are building a larger application or one which makes data available to multiple activities and/or applications, a content provider is the proscribed means of accessing your data.

- Broadcast receivers: You can launch an Android application to process a specific element of data or respond to an event, such as receiving a text message.

An Android application is deployed to a device along with a file named `AndroidManifest.xml`. This file is required for every Android application and is essentially the cheat sheet that tells the operating system exactly how to interact with your application. The `AndroidManifest.xml` includes both the required class names and types of events that the application is able to process as well as the required permissions that the application needs to run. For example, if an application requires access to the network to download a file, this permission must be explicitly stated in the manifest file. Or, perhaps an application needs to access the camera. The user needs to approve that also. This approach of declaratively telling the user what the application is going to use is essential in this day of increasing privacy and security concerns. While you can quickly just accept software licenses, it is important that you pay close attention to what an app is requesting to access.

Advantages of Android Development

- The Android is an open-source Operating system and hence possesses a vast community for support.
- The design of the Android Application has guidelines from Google, which becomes easier for developers to produce more intuitive user applications.
- Fragmentation gives more power to Android Applications. This means the application can run two activities on a single screen.
- Releasing the Android application in the Google play store is easier when it is compared to other platforms.

Disadvantages of Android

1. Low Security

Often, we see android apps getting breached or hacked and people's data getting leaked. However, at its best, Google is always coming up with security patches and resolving such issues.

2. Testing

Sometimes, testing an android app is ridiculous as its performance or state may vary from device to device and even at different Android versions.

3. Power Consumption

Android as an operating system consumes more power.

4. Lost Tracks

With the coming up of new android versions, it becomes pretty challenging to adapt yourself to new Android SDK tools.

5. Restrictions

Sometimes, there are restrictions on developers to follow the new standards for their existing applications. Failing to do so makes your existing app depreciated.

Applications of Android

You would be discussing various types of applications related to android. Android helps us to make our life easier and simpler. There are multiple kinds of applications solving numerous varieties of issues. Whether it's navigation, messaging, lifestyle, social media, weather, finance, food ordering, travel, games, etc.

Some of the most common applications are as follows:

1. Navigation

You have apps like Google Maps, which help us locate ourselves and find a route to visit the desired location.

2. Lifestyle

You have apps like Cult. Fit and Step Set Go helps you track your fitness, how many calories you are burning, and how many walking steps you are walking daily.

3. Messaging

There are apps like WhatsApp, Telegram, etc., which help us message any individual or a group of people together.

4. Travel

You have apps like MakeMyTrip, RedBus, etc., making our travel booking, hotel book, or other amenities very easy and at our fingertips.

How to change the Operating System of any Android device?

Did you ever think that you could install some other operating system instead of Android? Sometimes for some reason, you may need to change your operating system to some other.

There are various other operating systems in the market having their features and dependencies. Some of the most common operating systems are IOS, Symbian, Tizen, Kai OS, Chrome OS, Unix, Linux, Windows, etc.

Before proceeding to change your operating system, you should turn on the developer options on your device. First let's, have a look at the detailed steps of changing the operating system of your android device:

1: Move to Settings of your phone —> Click About Phone —> Click on Build Number 3-4 times you will get a toast saying, “you are now a developer.”

2: Now go to settings —> Developer Options —> Click on Enable USB Debugging.

3: Download the “Change My Software” tool from the below link
(<http://ow.ly/xTBl309o8fd>)

4: When the download is completed. Then connect your device with a USB cable.

Ensure that USB debugging is turned on.

5: Now open the Change My Software tool on your pc.

6: Now select android. Then it will prompt you to install the same tool on android.

Just confirm and proceed.

7: Click on continue and select the language if prompted.

8: Let the windows driver install. If any prompt is there, then confirm it and finally install.

9: You will locate an option called “Remove Android Just click on it..”

10: Once the installation is completed automatically, it will reboot your device.

Similarly, you can use this tool to revert to android or even convert your other Operating System based phone to an Android Device.

Challenges of Android Development

There can be several challenges that you can face while starting with Android App Development. These are easily trackable, and you can easily overcome these challenges by practicing more and developing more apps.

Many developers often face these challenges while developing apps for android. However, as beginners, we always tend to overlook or ignore

these challenging points and create a less secure app, does not fit the user's window or even crashes now and then.

To save yourself, you should always focus on the following points while working on Android.

1. Security

While developing an app, you should ensure that both user and system data are secure on the user's device.

2. Compatibility

You should develop an app that can work on most Android versions. Not all people have the latest versions of Android running on their devices; hence making a supportive app would help you target a more significant user section.

3. Performance

You should always try to develop an app that is responsive and doesn't lag. Many users don't have high-performance devices with greater RAM or memory space. So you should make sure that your app doesn't occupy much memory and doesn't create many junks.

4. Screen Variants

Your app should be adaptive enough to adapt to any screen size or resolution. Some people may run your app on their tablets or their wide-screen smartphones. So your app should fit into those screen variants.

5. Comments

You should always try to comment on your codes and work so that if anyone wants to add any feature in your application in the future, then they can easily understand what changes they need to make and where.

Future scope and importance of Android

Android is one of the widely used operating systems that require developers who can develop applications that can work on android. Therefore, it can be a future job market for the guys who know Android Development. With time, the number of users would increase in android, which will lead to a higher need for android developers.

Therefore, learning android may help you launch your career by creating your own company and reaching the maximum users globally with that app.

How to kick start with android easily?

There are not many prerequisites to start with Android. However, for any android developer, he should know some basic knowledge about what an android is, how it works and what architecture it has. Now comes which programming language you should study to proceed with android. Therefore, it depends on the IDE or framework you choose to work.

Nowadays, many IDEs (Integrated Development Environment) and tools help us develop apps for android. The most common being:

1. Android Studio
2. Flutter

Several other frameworks help us in building android apps like

1. React Native
2. Ionic

Among these, the most common and widely used is the Android Studio.

Android Studio gives you many features straight from designing your layouts to providing the functionalities. You would get an environment where you can write/debug your code and generate an APK published on Google Play Store.

Android Studio is one of the common IDEs for Android development. You can use it after gathering basic knowledge of XML and Java or Kotlin. On the other hand, Kotlin is one of the most popular general-purpose languages evolving in recent times.

Therefore, it is insisted on proceeding with Kotlin for the android development. This course will help you in understanding all aspects of android development using Android Studio.

Finally, we will use framework react native to build our app

What is React Native?

React Native is a JavaScript framework for writing real, natively rendering mobile applications for iOS and Android. It's based on React, Facebook's JavaScript library for building user interfaces, but instead of targeting the browser, it targets mobile platforms. In other words: web developers can now write mobile applications that look and feel truly "native," all from the comfort of a JavaScript library that we already know and love. Plus, because most of the code you write can be shared between platforms, React Native makes it easy to simultaneously develop for both Android and iOS.

Similar to React for the Web, React Native applications are written using a mixture of JavaScript and XML-esque markup, known as JSX. Then, under the hood, the

React Native "bridge" invokes the native rendering APIs in Objective-C (for iOS) or Java (for Android). Thus, your application will render using real mobile UI components, *not* webviews, and will look and feel like any other mobile application. React Native also exposes JavaScript interfaces for platform APIs, so your React Native apps can access platform features like the phone camera, or the user's location.

React Native currently supports both iOS and Android, and has the potential to expand to future platforms as well. In this book, we'll cover both iOS and Android. The vast majority of the code we write will be

cross-platform. And yes: you can really use React Native to build production-ready mobile applications! Some anecdota: Facebook, Palantir, and TaskRabbit are already using it in production for user-facing application

Advantages of React Native

The fact that React Native actually renders using its host platform's standard rendering APIs enables it to stand out from most existing methods of cross-platform application development, like Cordova or Ionic. Existing methods of writing mobile applications using combinations of JavaScript, HTML, and CSS typically render using webviews. While this approach can work, it also comes with drawbacks, especially around performance. Additionally, they do not usually have access to the host platform's set of native UI elements. When these frameworks do try to mimic native UI elements, the results usually “feel” just a little off; reverse-engineering all the fine details of things like animations takes an enormous amount of effort, and they can quickly become out of date.

In contrast, React Native actually translates your markup to real, native UI elements, leveraging existing means of rendering views on whatever platform you are working with. Additionally, React works separately from the main UI thread, so your application can maintain high performance without sacrificing capability. The update cycle in React Native is the same as in React: when props or state change, React Native re-renders the views. The major difference between React Native and React in the browser is that React Native does this by leveraging the UI libraries of its host platform, rather than using HTML and CSS markup.

For developers accustomed to working on the Web with React, this means you can write mobile apps with the performance and look and feel of a native application, while using familiar tools. React Native also represents an improvement over normal mobile development in two other areas: the developer experience and cross-platform development potential.

Introduction of Artificial intelligence

A Brief History of AI

The concept of Artificial Intelligence is not as modern as we think it is. This traces back to as early as 1950 when Alan Turing invented the Turing test. Then the first chatbot computer program, ELIZA, was created in the 1960s. [1] IBM deep blue was a chess computer made in 1977 beat a world chess champion in two out of six games, one won by the champion and the other three games were draw. [2] In 2011, Siri was announced as a digital assistant by Apple. [3] Elon Musk and some others founded Open AI in 2015.

A Brief History of Deep Learning:

The history of deep learning can be traced back to 1943, when Walter Pitts and Warren McCulloch created a computer model based on the neural networks of the human brain.

They used a combination of algorithms and mathematics they called “threshold logic” to mimic the thought process. Since that time, Deep Learning has evolved steadily, with only two significant breaks in its development. Both were tied to the infamous Artificial Intelligence winters.

The 1960s:

Henry J. Kelley is given credit for developing the basics of a continuous Back Propagation Model in 1960. In 1962, a simpler version based only on the chain rule was developed by Stuart Dreyfus. While the concept of back propagation (the backward propagation of errors for purposes of training) did exist in the early 1960s, it was clumsy and inefficient, and would not become useful until 1985.

The earliest efforts in developing deep learning algorithms came from Alexey

Grigoryevich Ivakhnenko (developed the Group Method of Data Handling) and

Valentin Grigor'evich Lapa (author of Cybernetics and Forecasting Techniques) in 1965. They used models with polynomial (complicated equations) activation functions, that were then analyzed statistically. From each layer, the best statistically chosen features were then forwarded on to the next layer (a slow, manual process).

The 1970s:

During the 1970's the first AI winter kicked in, the result of promises that couldn't be kept. The impact of this lack of funding limited both DL and AI research.

Fortunately, there were individuals who carried on the research without funding.

The first “convolutional neural networks” were used by Kunihiro Fukushima. Fukushima designed neural networks with multiple pooling and convolutional layers. In 1979, he developed an artificial neural network, called Neocognitron, which used a hierarchical, multilayered design. This design allowed the computer the “learn” to recognize visual patterns. The networks resembled modern versions, but were trained with a reinforcement strategy of recurring activation in multiple layers, which gained strength over time. Additionally, Fukushima's design allowed important features to be adjusted manually by increasing the “weight” of certain connections.

Many of the concepts of Neocognitron continue to be used.

The use of top-down connections and new learning methods have allowed for a variety of neural networks to be realized. When more than one pattern is presented at the same time, the Selective Attention Model can separate and recognize individual patterns by shifting its attention from one to the other. (The same process many of us use when multitasking). A modern Neocognitron can not only identify patterns with missing information (for example, an incomplete number 5), but can

also complete the image by adding the missing information. This could be described as “inference.”

Back propagation, the use of errors in training deep learning models, evolved significantly in 1970. This was when Seppo Linnainmaa wrote his master’s thesis, including a FORTRAN code for back propagation.

Unfortunately, the concept was not applied to neural networks until 1985. This was when Rumelhart, Williams, and Hinton demonstrated back propagation in a neural network could provide “interesting” distribution representations. Philosophically, this discovery brought to light the question within cognitive psychology of whether human understanding relies on symbolic logic (computationalism) or distributed representations (connectionism).

The 1980s and 90s:

In 1989, Yann LeCun provided the first practical demonstration of backpropagation at Bell Labs. He combined convolutional neural networks with back propagation onto read “handwritten” digits. This system was eventually used to read the numbers of handwritten checks.

This time is also when the second AI winter (1985-90s) kicked in, which also effected research for neural networks and deep learning. Various overly-optimistic individuals had exaggerated the “immediate” potential of Artificial Intelligence, breaking expectations and angering investors. The anger was so intense, the phrase Artificial Intelligence reached pseudoscience status. Fortunately, some people continued to work on AI and DL, and some significant advances were made. In 1995, Dana Cortes and Vladimir Vapnik developed the support vector machine (a system for mapping and recognizing similar data). LSTM (long short-term memory) for recurrent neural networks was developed in 1997, by Sepp Hochreiter and Juergen Schmidhuber.

The next significant evolutionary step for deep learning took place in 1999, when computers started becoming faster at processing data and GPU (graphics processing units) were developed. Faster processing, with GPUs processing pictures, increased computational speeds by 1000 times over a 10-year span. During this time, neural networks began to compete with support vector machines. While a neural network could be

slow compared to a support vector machine, neural networks offered better results using the same data. Neural networks also have the advantage of continuing to improve as more training data is added.

2000-2010:

Around the year 2000, The Vanishing Gradient Problem appeared. It was discovered “features” (lessons) formed in lower layers were not being learned by the upper layers, because no learning signal reached these layers. This was not a fundamental problem for all neural networks, just the ones with gradient-based learning methods. The source of the problem turned out to be certain activation functions. A number of activation functions condensed their input, in turn reducing the output range in a somewhat chaotic fashion. This produced large areas of input mapped over an extremely small range. In these areas of input, a large change will be reduced to a small change in the output, resulting in a vanishing gradient. Two solutions used to solve this problem were layer-by-layer pre-training and the development of long short-term memory.

In 2001, a research report by META Group (now called Gartner) described the challenges and opportunities of data growth as three-dimensional. The report described the increasing volume of data and the increasing speed of data as increasing the range of data sources and types. This was a call to prepare for the onslaught of Big Data, which was just starting.

In 2009, Fei-Fei Li, an AI professor at Stanford launched ImageNet, assembled a free database of more than 14 million labeled images. The Internet is, and was, full of unlabeled images. Labeled images were needed to “train” neural nets. Professor Li said, “Our vision was that big data would change the way machine learning works. Data drives learning.”

2011-2020:

By 2011, the speed of GPUs had increased significantly, making it possible to train convolutional neural networks “without” the layer-by-layer pre-training. With the increased computing speed, it became

obvious deep learning had significant advantages in terms of efficiency and speed. One example is AlexNet, a convolutional neural network whose architecture won several international competitions during 2011 and 2012. Rectified linear units were used to enhance the speed and dropout.

Also in 2012, Google Brain released the results of an unusual project known as The Cat Experiment. The free-spirited project explored the difficulties of “unsupervised learning.” Deep learning uses “supervised learning,” meaning the convolutional neural net is trained using labeled data (think images from ImageNet). Using unsupervised learning, a convolutional neural net is given unlabeled data, and is then asked to seek out recurring patterns.

The Cat Experiment used a neural net spread over 1,000 computers. Ten million “unlabeled” images were taken randomly from YouTube, shown to the system, and then the training software was allowed to run. At the end of the training, one neuron in the highest layer was found to respond strongly to the images of cats.

Andrew Ng, the project’s founder said, “We also found a neuron that responded very strongly to human faces.” Unsupervised learning remains a significant goal in the field of deep learning.

The Generative Adversarial Neural Network (GAN) was introduced in 2014. GAN was created by Ian Goodfellow. With GAN, two neural networks play against each other in a game. The goal of the game is for one network to imitate a photo, and trick its opponent into believing it is real. The opponent is, of course, looking for flaws. The game is played until the near perfect photo tricks the opponent. GAN provides a way to perfect a product (and has also begun being used by scammers).

The Future of Deep learning and Business:

Deep learning has provided image-based product searches – Ebay, Etsy– and efficient ways to inspect products on the assembly line. The first supports consumer convenience, while the second is an example of business productivity.

Currently, the evolution of artificial intelligence is dependent on deep learning. Deep learning is still evolving and in need of creative ideas.

Semantics technology is being used with deep learning to take artificial intelligence to the next level, providing more natural sounding, human-like conversations.

Banks and financial services are using deep learning to automate trading, reduce risk, detect fraud, and provide AI/chatbot advice to investors. A report from the EIU (Economist Intelligence Unit) suggests 86% of financial services are planning to increase their artificial intelligence investments by 2025.

Deep learning and artificial intelligence are influencing the creation of new business models. These businesses are creating new corporate cultures that embrace deep learning, artificial intelligence, and modern technology.

Towards Conclusion...

The growth of Artificial Intelligence in recent times has been exponential. We cannot even imagine how big, impactful AI is going to be in the near future, and how drastically it is going to change and upgrade the world we live in today. There are a lot more to learn about AI and its rapidly growing applications in our life. I believe it would be wise to adapt to this changing world and acquire skills related to Artificial Intelligence and technology. Just like AI learns and develops, we should too - to make this world a better place.

Introduction to Data

Data is a powerful tool that can be used to inform decision-making, identify patterns and trends, measure performance and progress, and gain insights into the future. It plays an important role in many fields and industries, including business, finance, healthcare, education, and science.

Through data collection and analysis, organizations and individuals can gain a deeper understanding of their operations, customers, and markets. This understanding can then be used to identify opportunities for improvement, optimize performance, and make data-driven decisions. Data can also be used to measure key performance indicators (KPIs) and assess progress against established goals and benchmarks.

Data can also be used to gain insights and make predictions about future trends and patterns. With the increasing availability of large data sets and advances in analytics and machine learning, organizations and individuals can now harness the power of data to gain a competitive edge and stay ahead of the curve. For example, by analyzing customer data, businesses can identify patterns in consumer behavior and develop targeted marketing strategies. In healthcare, data can be used to identify trends in disease patterns and inform the development of treatment plans.

The importance of data cannot be overstated. It is critical for organizations to have accurate and up-to-date data in order to make informed decisions and stay competitive in today's fast-paced business environment. In addition, data is essential for evaluating and improving the performance of organizations and systems. It enables organizations to identify areas for improvement and track the impact of changes and interventions.

Data is also important for research and development. In many fields, data is used to test hypotheses and make discoveries that lead to new products, services, and technologies. Data can be used in both quantitative and qualitative form, and can be collected from a variety of sources, such as surveys, experiments, or observational studies.

Data is a vital aspect of any project, as it provides the information necessary for decision-making and problem-solving. Data can take many forms, such as numerical, text, or visual, and can be collected from a variety of sources, such as surveys, experiments, or observational studies. The importance of data lies in its ability to help organizations and individuals make informed decisions, identify patterns and trends, and measure performance and progress.

In conclusion, data is an essential resource that can be used to inform decisionmaking, identify patterns and trends, measure performance and progress, and gain insights into the future. Its importance cannot be overstated, as it is vital for organizations and individuals to have accurate and up-to-date data in order to make informed decisions and stay competitive in today's fast-paced business environment.

Importance of data in our system:

In the context of a parking app, data plays a crucial role in providing users with real-time information about available parking spaces, as well as in managing and optimizing the overall parking system. The app uses data to provide users with information such as the location, availability, and cost of parking spaces, and can also use data to manage the parking system, such as by monitoring usage patterns and identifying areas where additional parking is needed.

One important aspect of the parking app is the use of data to provide real-time information about available parking spaces. This information can be collected from a variety of sources, such as sensors in parking garages and lots, or from users of the app who update the availability of parking spaces in real-time. By providing users with real-time information about available parking spaces, the app can help them find a parking space

quickly and easily, reducing the amount of time and fuel wasted searching for a parking spot.

Another important aspect of the parking app is the use of data to manage and optimize the overall parking system. This can include monitoring usage patterns to identify areas where additional parking is needed or using data to identify patterns of overuse or misuse of parking spaces. By using data to manage and optimize the parking system, the app can help to ensure that parking spaces are being used efficiently and effectively, and that the system is able to meet the needs of users.

Data can also be used in the parking app to improve the user experience. For example, the app can use data on users' preferences and behavior to make personalized recommendations for parking spaces and to offer discounts and promotions to certain users. The app can also use data to learn users' habits and patterns of parking, and to offer them a more personalized and convenient parking experience.

In addition, data can also be used for monetization purposes. The app can use data to provide targeted advertising and offers to users, or to offer paid services such as reserved parking or valet services.

In conclusion, data plays a critical role in a parking app, providing users with realtime information about available parking spaces, managing and optimizing the overall parking system, personalizing the user experience, and also for monetization purposes. In your book, you can discuss these points in more detail and provide examples of how the app uses data to achieve these objectives and how it can benefit the users.

Introduction to Embedded System

System:

An arrangement in which all its unit assemble work together according to a set of rules. It can also be defined as a way of working, organizing or doing one or many tasks according to a fixed plan. For example, a watch is a time displaying system. Its components follow a set of rules to show time. If one of its parts fails, the watch will stop working. So, we can say, in a system, all its subcomponents depend on each other.

Embedded System

A combination of computer hardware and software where software is embedded into hardware is called an embedded system. It is an electronic system that can be programmed based on our needs. There are a set of protocols where we can organize the way of working and controlling the system. The hardware where the software is embedded is built with the help of a microcontroller. The hardware is comprised of a user interface, which includes the display, input, and/or output interfaces and memory of the system. When we consider types of Embedded Systems, power supply, processor, and communication ports are needed for the system to function properly.

Structure of embedded systems

Embedded systems vary in complexity but, generally, consist of three main elements:

- **Hardware.** The hardware of embedded systems is based around microprocessors and microcontrollers. Microprocessors are very similar to microcontrollers and, typically, refer to a CPU (central processing unit) that is integrated with other basic computing components such as memory chips and digital signal processors (DSPs). Microcontrollers have those components built into one chip.
- **Software and firmware.** Software for embedded systems can vary in complexity. However, industrial-grade microcontrollers and

embedded IoT systems usually run very simple software that requires little memory.

- Real-time operating system. These are not always included in embedded systems, especially smaller-scale systems. RTOSes define how the system works by supervising the software and setting rules during program execution.

Basic Structure of an Embedded System

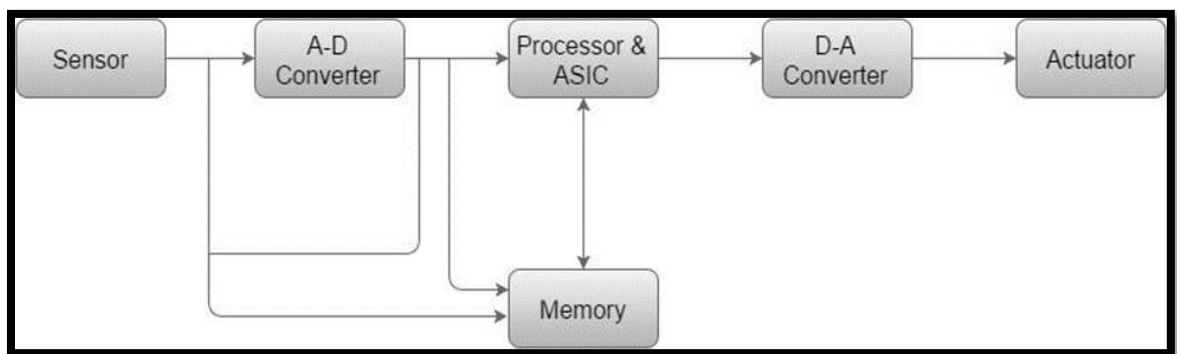


Figure 4 : Basic structure of an embedded system

Embedded Systems Structure

- Sensor – It measures the physical quantity and converts it to an electrical signal which can be read by an observer or by any electronic instrument like an A2D converter. A sensor stores the measured quantity to the memory.
- A-D Converter – an analog-to-digital converter converts the analog signal sent by the sensor into a digital signal.
- Processor & ASICs – Processors process the data to measure the output and store it to the memory.
- D-A Converter – A digital-to-analog converter converts the digital data fed by the processor to analog data

- Actuator – An actuator compares the output given by the D-A Converter to the actual (expected) output stored in it and stores the approved output.

Types of Embedded Systems

We can classify embedded systems based on performance and functional requirements and based on the performance of the microcontroller.

- Based on performance and functional requirements:

Stand-alone Embedded Systems

This type of embedded systems, as the name suggests, does not require a host system like a computer or a processor as it works by itself and displays data on the connected device or make necessary changes on the device. Input data is taken from the ports as analog or digital signals, and processing is done in the port itself. The result after proper calculation and conversion is displayed through a connected device. This result may either control or drive or display the device depending on the data. These systems offer flexibility and efficiency even though they work alone. Various examples are washing machines, mobile phones, or any systems that work alone without a computer's help.

Real-time Embedded Systems

When an output is required at a particular time, real-time embedded systems can be used. When a task has to be completed on time or a deadline for a project, this system comes as a savior. An external environment is controlled with the help of computer systems and connected through sensors or any other output/input interfaces. We can schedule the output either through a static or dynamic manner. There are two types under this category. They are soft and hard real-time embedded systems.

Soft Embedded Systems: These systems consider processes as the main task and manage the task completely. Deadlines are not considered as a priority, and even if the deadlines are missed, missing the processes should not happen in this system. In a computer system, the sound system is an example of a soft system. But it should be noted that deadlines should not always be missed as it results in the system's degradation.

Hard Embedded Systems: These systems consider timelines as the deadline, and they should not be missed in any case. Also, even if there are any delays, they should also be time-bound in the system. There is no permanent memory, and hence the processes should be done properly for the first time itself. The best example of a hard embedded system is an aircraft control system. The timeline should not be missed as well as the processes.

Network Embedded Systems

When a program is running inside another device, a network is formed. This is called network embedded systems, where a microprocessor or a controller controls the running program. A network is related to this system, and they can be either LAN or WAN. It is not necessary that the connection should be wired or wireless. This category can be considered the fastest growing in the embedded systems due to flexibility and connection. Also, there is a web server where the connection is based upon the web browser. All the network is controlled and accessed with the help of a web browser. Security systems in any office or tech park are examples of network embedded systems where all the connections are made through a common network and controlled under one umbrella.

Mobile Embedded Systems

All the devices that are portable and working with an embedded system is a mobile embedded system. Though there is a limitation of memory and functionality, its portability and handy systems are useful for all people. The best example that we can connect easily is mobile phones, laptops, and calculators.

- Based on the performance of the microcontroller:

Small Scale Embedded Systems: 8 bit or 16-bit microcontrollers are used to design these and work with the help of batter in the system. Several programming tools are used to develop small scale embedded systems. The hardware is very small, and the processor is slow. The memory is also less. The codes for developing these embedded systems can be written with the help of any IDE.

Medium Scale Embedded Systems: 16 bit or 32-bit microcontrollers are used to develop medium systems. Also, these can be developed with DSPs or RISCs. Hardware and software functionalities are complex, and several coding languages can be used as programming tools. As small-scale systems, an IDE is required for medium scale systems also. We can use medium-scale systems in high-end applications with large memory and processing data.

Sophisticated Embedded Systems: The most complex embedded system with all the difficult complexities of hardware and software that makes the system useful for all is called sophisticated embedded systems. These systems require registers of huge memory, scalable processors, and IPs to work well in any environment. They are used in systems with graphical screens, touchpads, and cutting-edge options where software and hardware are equally needed for performance.

Advantages and Disadvantages of Embedded System:

Advantages:

- Small size.
- Enhanced real-time performance.
- Easily customizable for a specific application.

Disadvantages:

- High development cost.
- Time-consuming design process.
- As its application-specific less market available.

Chapter (2) Background

In this chapter, we will explore the essential components of documenting a project, providing a comprehensive overview of the project's concept, requirements, and feasibility.

Every successful project begins with a clear understanding of its purpose and objectives. To establish this understanding, we will start with the Abstract, which presents a concise summary of the project idea and its significance. The abstract will provide a glimpse into the project's scope, goals, and potential benefits, setting the stage for the detailed exploration that follows.

Next, we will delve into the Survey section, which serves as an extensive review of related projects, technologies, and research. Conducting a thorough survey helps identify existing solutions, understand the current state of the field, and gain insights into the strengths and limitations of similar projects. The survey section will provide a foundation for the project by highlighting the unique aspects and contributions it brings to the table.

Following the survey, we will move on to the Requirements section, where we define both the functional and non-functional requirements of the project. Functional requirements describe the specific functionalities and features the project should provide, while non-functional requirements encompass aspects such as performance, security, usability, and scalability. By clearly defining these requirements, we ensure that the project meets the needs and expectations of its users.

Additionally, we will discuss the Database section, which focuses on the design and implementation of the project's data storage and management. A well-designed database is crucial for efficient data organization, retrieval, and manipulation. We will explore different database models, schema design, and data storage technologies, highlighting their relevance to the project and its objectives.

Lastly, we will conduct a Feasibility Study to assess the practicality and viability of the project. This study examines factors such as technical feasibility,

economic feasibility, and operational feasibility to determine whether the project is realistic and achievable within the given constraints. The feasibility study provides valuable insights into the potential challenges, risks, and benefits associated with the project.

Throughout this chapter, we will provide guidance and examples to help you effectively document your project. Whether you are a student working on an academic project or a professional involved in a real-world endeavor, the concepts and techniques covered in this chapter will assist you in presenting a well-structured and comprehensive project documentation.

Abstract

The ever-increasing population has led to chaotic city traffic. As a result of the process of searching for a parking lot becomes tedious. It is time consuming task leading to discomfort. The fuel consumption is on an increasing side due to such scenarios. The increase in vehicular traffic creates a negative impact on the environment. In the wake of smart city

Times these issues, lead us to the need of a „smart“ solution. To resolve these issues and satisfy the increasing demand for the parking areas, parking management organizations are trying to implement better and technologically advanced solutions. A smart parking application will enable real time parking availability monitoring and reservation thereby providing better services to the end users as well as reduce the workload of the parking administrator.

The system has two types of users (client and parking owner).

The system goal is to make parking operation easy by using mobile app to determine the nearest parking, determine available car parking area, and manage the parking automatically by using cameras and sensors.

Survey

1. SpotHero

Once known as Parking Panda, Spot Hero is a parking reservation service that works with different facilities around the country to provide customers with parking in 28 major cities. Customers can find, reserve, and pay for a parking spot in a lot or garage — often at a discounted rate — and receive a parking pass either via email or through the app.

This service helps drivers find and reserve hourly, monthly and airport parking, as well as parking for events, through partnerships with sporting venues ranging from Dodgers Stadium in Los Angeles to Yankee Stadium in New York.

2. Best Parking

Best Parking works by letting users search for parking in the moment or for future trips. Drivers can reserve spaces ahead of time, get a digital parking pass, and save as much as 50%. Best Parking links directly to the mobility platform Arrive. Best Parking was acquired in 2015 by Park Whiz, the next entry on our list.

3. ParkWhiz

Founded in 2006, ParkWhiz works with 4,000 parking operators in 35 states and 50 cities. Customers can use the service to find places to park on a daily or monthly basis, or for events, compare prices, and reserve parking spaces.

ParkWhiz also operates ParkWhiz Business, an online system that helps companies manage their employee and customer parking.

4. Parkopedia

Operating in 89 countries — and keeping tabs on more than 70 million parking spots, Parkopedia could become your favorite navigation app. Just tell Parkopedia where you're headed in the city of your choice and it will find the closest spot for your car, how much it will cost, and how many spaces are

available. The company bills itself as the “parking partner” of a number of carmakers, including GM, Ford, Mazda, and Volvo.

5. Way

Technically, Way is more than just an app for parking, as the company offers insurance and auto finance services. When it comes to helping people find parking spaces, Way lets users reserve hourly and monthly parking and helps locate parking spaces at the airport.

Way also offers a service called Parking Pass, which lets drivers save money, letting them hop from lot to lot for a one-time daily or monthly price. And if you’re not sure where you’re parked, Way will provide walking directions back to your lot.

6. SpotAngels

The founder of SpotAngels built their app after having their car towed and being forced to pay \$569 to recover it. Available in almost every major U.S. city, the app helps people find street parking and avoid tickets, showing users parking options, and sending reminders to help drivers avoid tickets. The company website even has a map to help users find free parking.

7. ParkMobile

This free app offers three different ways to park, but for our purposes, we’ll focus on just one: the ability to reserve a parking space in advance. With ParkMobile, drivers who are heading into a city for the day and need to find a parking spot can search for parking garage facilities. Simply select the garages you want and reserve a spot and a time for their car.

8. The Parking Spot

The Parking Spot is in the conversation for the best airport parking apps, as it allows you to find places to park at a number of major airports around the U.S. The app lets users pay with touchless entry, and in some locations, offers complimentary car cleaning while travelers are away.

9. PayByPhone

Owned by Volkswagen Financial Services, this app lets users pay for street parking by entering their location on their phone, based on a designated number provided by the PayByPhone system on a nearby sign. Drivers can enter the amount of time they plan to stay and extend their stay without having to run back to their car to feed the meter.

10. Passport Parking

Much like PayByPhone, Passport Parking lets drivers find and reserve parking and pay for their spots by entering a location number on the app and paying from their phone. This app notifies users when their session is about to end, as well as when parking rates will change.

What they provide more than us:

-they provide street slot for parking, use can find the nearest empty space in street

Advantages we has:

- We has a full automated sys from the start of reservation to the end
- They has a limited slots because they has only there own slots , we don't mind that any garage work with us so that leading to availability every where

Future Work:

Provide street slots for parking

Function Requirements:

Firstly, with respect to user (car owner):

1. System will allow every user to make registration via the mobile app by adding his data as (first name, middle name, Last name, profile picture, email, password, phone number) on the other hand, with social accounts.
2. User can make login with username, email, and password or with social accounts.
3. User can update or delete his data later (his account).
4. System's server will relate to google maps to determine parking locations and show it to users from the app.
5. System will allow to users to search via google maps for the nearest parking from his location and choose it.
6. System will show number of free car areas and price per hour for every parking.
7. System allow user to reserve in the parking that he wants from his mobile.
8. System will show timer on the app for user after his reservation.
9. System (mobile app) will generate QR code to every user after reservation
10. User should enter his vehicle's plat number to reserve and get QR code to enter the parking.
11. User can make monthly or weakly reservation (Participation).
12. User maybe get discount on his reservation.
13. User can cancel his reservation after certain time only or take a fine.

14. System will store a picture of car driver and will store car color and model for every car will enter the parking.
15. User will scan his QR code to get into the parking
16. System will count every car enter parking and determine number of free car areas in the parking.
17. There are baffles on every car area to prevent any user tries to park in one other his one.
18. These baffles will open only to the user who allowed to him to park in it.
19. System will reserve a car area for every user after scanning QR code to park his car in it.
20. System will show user his car area on the app (row and column) and no of floor to park in it.
21. System will determine every car area if its busy or not.
22. Every car area will be locked and will open only when the user arrives to park.
23. User will scan his QR code again to get out from the parking
24. System allows to user to share his QR code to allow any person has this QR code to get out the car from the parking.
25. System will determine user parking time and money.
26. User can pay online, credit or cash.
27. User can rate the service.

Then with respect to parking owner:

- 28. The system will give parking owner dashboard to manage his parking.
- 29. Parking owner can register on the system and enter his data as (email, username, and password) to manage his parking.
- 30. Parking owner will enter the parking data on the system as (Parking name, location, price per hour, number of floors, number of rows, number of columns, number of parking areas).
- 31. Parking owner can login to the system with username, email, and password.
- 32. Parking owner can update or delete his data.
- 33. Parking owner can see the number of vehicles in today and out today.
- 34. Parking owner can see total income.
- 35. Parking owner can make reports.

Then with respect to admin

- 36. Admin manage all accounts of (users and parking owners)
- 37. Admin have dashboard build on BI for all data of the system)

Non-function requirements

1- Security

The system should provide confidentiality for user data using database encryption and local encryption to protect data in the event of device theft (laptop/handheld device).

2- Performance

- No delays in the application.
- Fast data processing.
- Fast system training.

3- Portability

- The central repository should be platform independent so that it can be accessible and store application data via the web application and the mobile app.
- The android application provides high availability and high accuracy in finding the locations.

4- Reliability

The system must perform without failure in 95 percent of use cases during a month.

5- Maintainability

The mean time to restore the system following a system failure must not be greater than 10 minutes.

6- Availability

- The application must be available to use 24 hours.
- The server should be able to handle concurrent requests from different users.

7- Usability

- Easy to use.
- Simple booking process.

Database

Entity Relationship Diagrams (ERD)

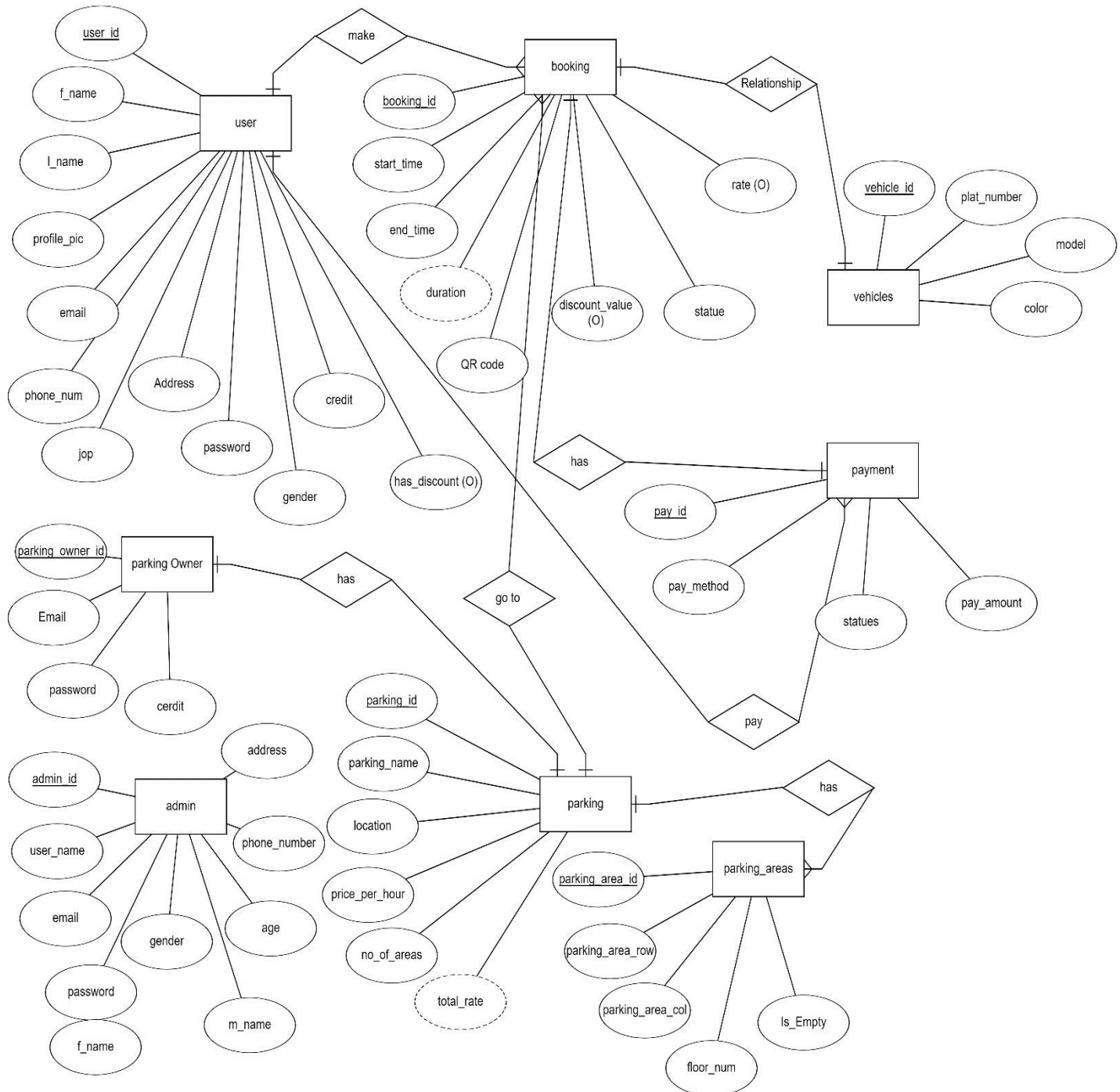


Figure 5: ERD

Schema

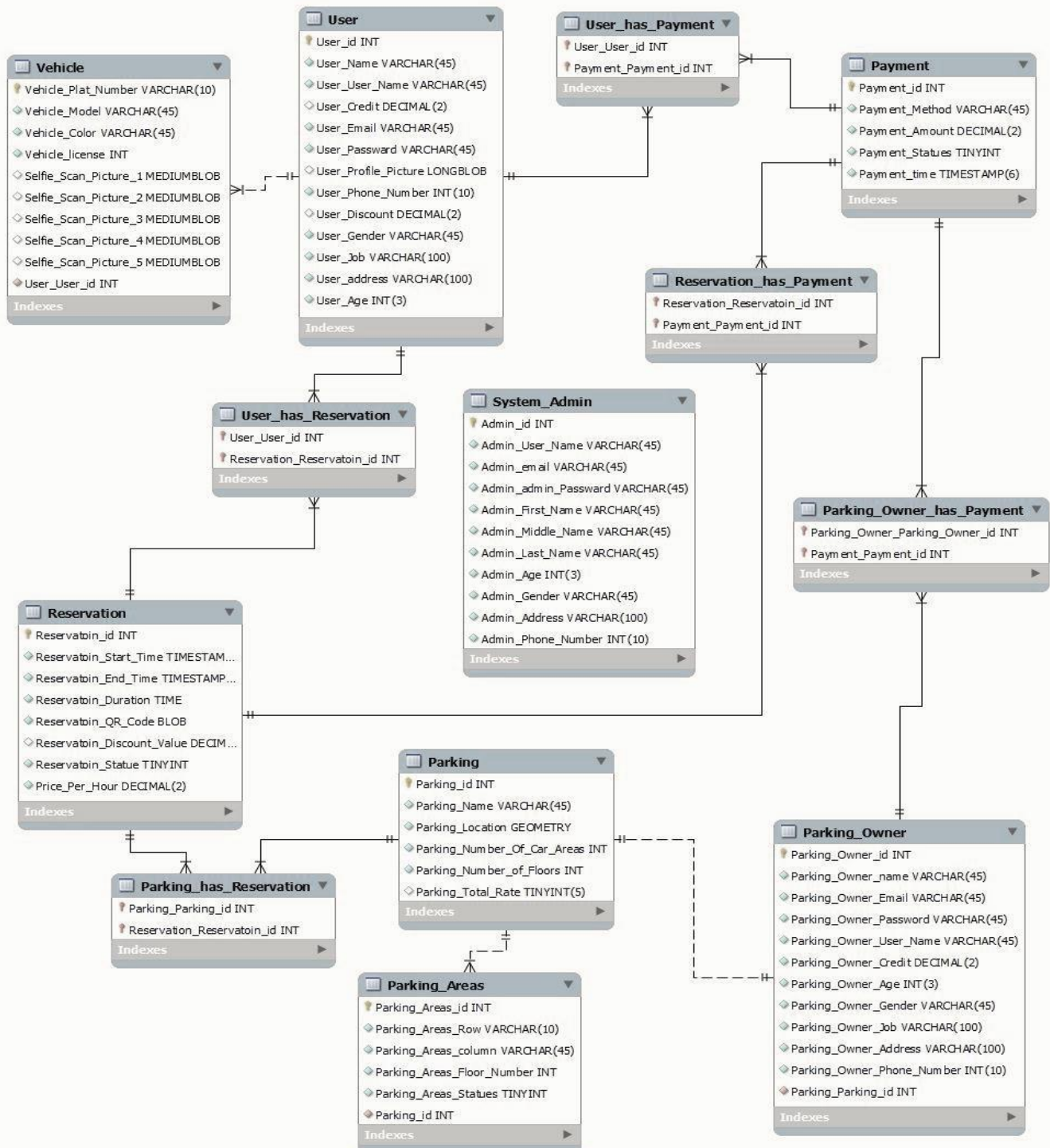


Figure 6: Schema

Business Model

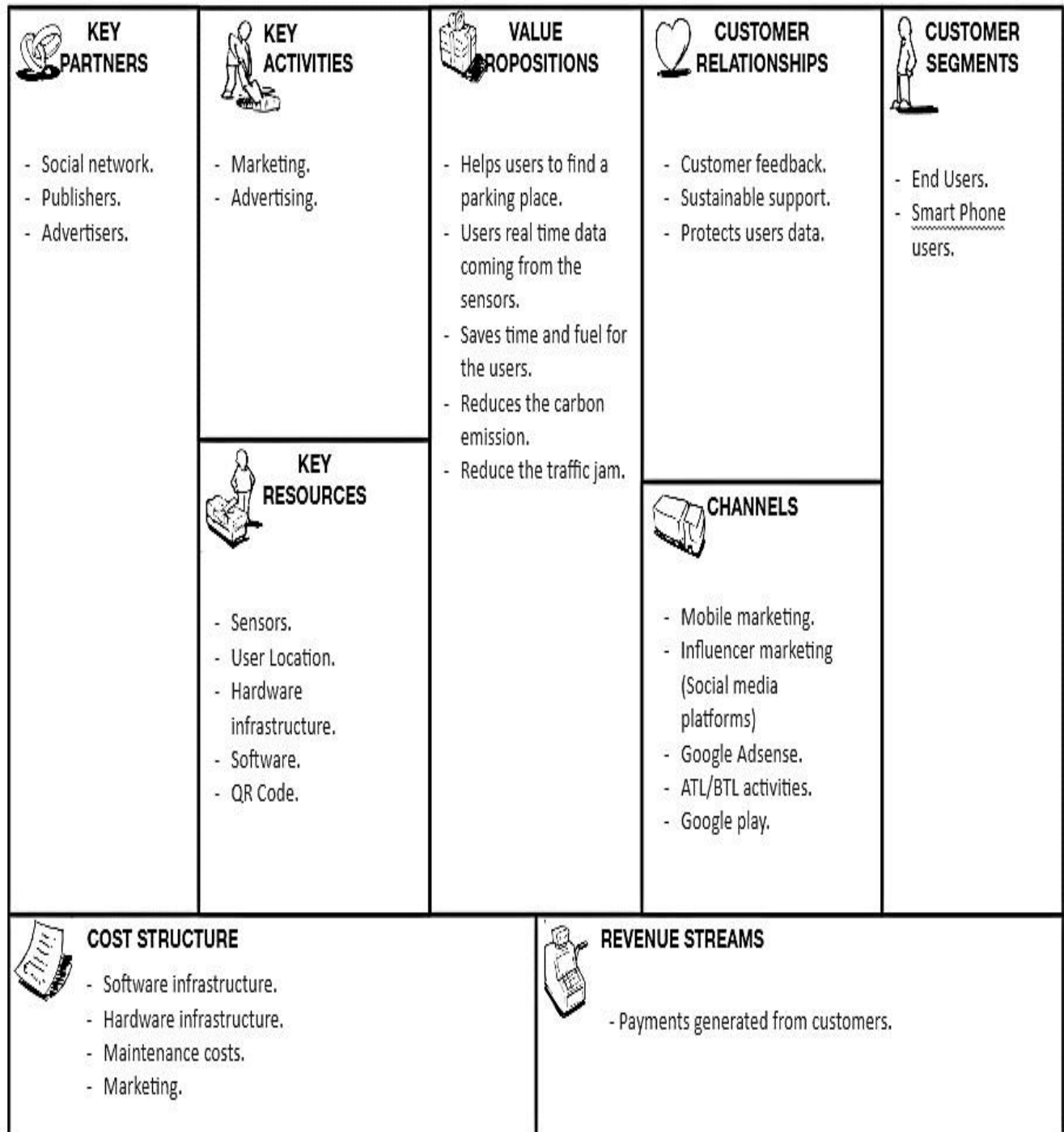


Figure 7: Business Model

Feasibility study

Technical Feasibility

Project “Rakna” is a complete mobile application and web application link with embedded system and Artificial intelligence. The main technologies and tools that are associated with “Rakna” are:

- Hard ware:
Laptop, Arduino Uno or Atmega32, Stepper motor, Stepper motor driver, IR sensor, Infrared sensor, LCD display, Ultrasonic sensor, Jumpers, Cam module, Power supply, GSM module
- Software:
Micro C, Extreme burner, Rufus, Proteus
- Languages:
Embedded C, HTML, css, JavaScript, React native, React js, PHP, laravel , Mysql

To apply our project in real life, we have default general budget;

Garage has some floors ;the garage was built on 5,000 square meters; the storage capacity of the project is 850.

Every floor has a Camera in every corner and I will explain the cost of row of cars (20 cars) all link with original embedded system, each of car area have sensor and block.

Economic feasibility

We will allow to our users (Car owner and Parking owner) to get access on our system and our company will have commission on every parking process

Chapter (3) Design

Welcome to the chapter on designing our project using UML Use Case diagrams and descriptions! In this chapter, we will dive into the process of designing our project by employing UML (Unified Modeling Language) techniques. Specifically, we will focus on UML Use Case diagrams and Use Case descriptions, which are powerful tools for capturing the functional requirements and interactions within our system.

Design is a critical phase in software development that transforms project requirements into a structured blueprint. UML Use Case diagrams and descriptions help us visualize and describe the various user interactions and system behaviors, providing a foundation for effective development and implementation.

Section 1: UML Use Case Diagrams

In this section, we will explore UML Use Case diagrams, which provide a graphical representation of the interactions between actors and the system. Use Case diagrams help us understand the system's functionality from a user's perspective and identify key actors and their relationships with the system. We will learn how to create Use Case diagrams, define actors, and depict use cases and their associations.

Section 2: Use Case Descriptions

In this section, we will delve into Use Case descriptions, which offer detailed textual explanations of each use case identified in the Use Case diagram. Use

Case descriptions capture the flow of events, preconditions, postconditions, and other relevant information about each use case. We will learn how to write clear and concise Use Case descriptions that effectively document the steps and interactions involved in executing each use case.

Section 3: Applying Use Case Diagrams and Descriptions

In this section, we will apply the knowledge gained in the previous sections to design our project. We will walk through the process of creating a Use Case diagram that accurately represents the system's functionality and the interactions with various actors. We will then develop comprehensive Use Case descriptions for each identified use case, capturing the necessary details to guide the implementation phase.

By utilizing UML Use Case diagrams and descriptions, we establish a shared understanding of the project's requirements, facilitating effective communication among stakeholders, developers, and designers. These tools enable us to visualize and describe the system's functionality, identify potential interactions, and ensure that the final implementation aligns with the intended objectives.

Throughout this chapter, we will provide practical examples and guidelines to help you design your project using UML Use Case diagrams and descriptions. By the end of this chapter, you will have a solid foundation in leveraging these tools to create a well-structured and user-centric design for your project. Let's embark on this exciting journey of designing our project!

Use case diagram

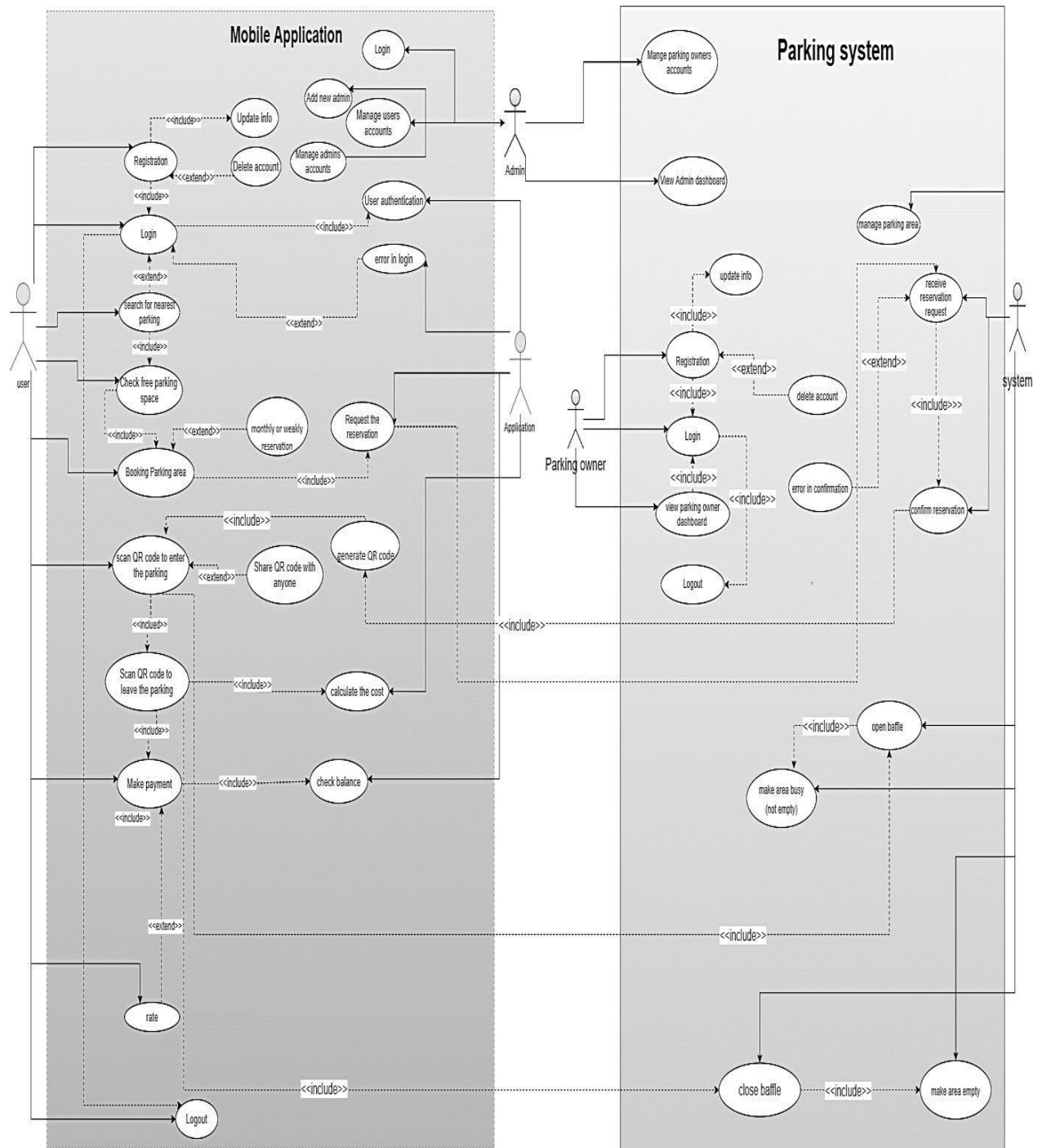


Figure 8: Use case diagram

Use case description

Table 1: Admin login

Use case name : Admin login	
Use case number	Uc-01
Use case description	This function help the admin to login to enter his dashboard
Actor	Admin – system
Pre-condition	The person how try to enter is must already his role is admin
Sequence of event	- Admin open login page - Enter his user name or email - Enter his password - Click login button - Dashboard of admin will opened
Post condition	Admin can manage the system
Exceptions	- Admin try wrong user name - Admin try wrong password - User is not admin

Table 2: Add new admin

Use case name : Add new admin	
Use case number	Uc-02
Use case description	This function help super-admin to add new admins
Actor	Admin – system
Pre-condition	This user not admin
Sequence of event	- Admin click add new admin - Admin add the username of new admin - Admin add the email of new admin - Admin add the password of new admin Press save button
Post condition	New admin added
Exceptions	Admin add wrong or invalid data

Table 3: View user data

Use case name: View user data	
Use case number	Uc-03
Use case description	This function help user to view users data
Actor	Admin - system
Pre-condition	Admin need to view the data of the users
Sequence of event	- Admin login - Admin click on mange users - Table of users appear
Post condition	Data of user fetched
Exceptions	- Lose connection with database

Table 4: Block user

Use case name: Block user	
Use case number	Uc-04
Use case description	This function help admin to block user from the system
Actor	Admin – system
Pre-condition	Admin need to block user
Sequence of event	- Admin click mange users button - Admin choose the user - Admin click on block button
Post condition	This user blocked
Exceptions	- Lose connection

Table 5: View admin data

Use case name: View admin data	
Use case number	Uc-05
Use case description	This function help super-admin to view the data of admins
Actor	Admin – system
Pre-condition	Admin needed to see the data of all admins
Sequence of event	- Admin login - Admin click on mange admins - Table of admin appear
Post condition	Data of admins fetched
Exceptions	Lose connection with database

Table 6: Block admin

Use case name: Block admin	
Use case number	Uc-06
Use case description	This function help admin to block admin from the system
Actor	Admin – system
Pre-condition	Admin need to block admin
Sequence of event	- Admin click mange admins button - Admin choose the admin - Admin click on block button
Post condition	admin blocked
Exceptions	Lose connection

Table 7: View parking owner data

Use case name: View parking owner data	
Use case number	Uc-07
Use case description	This function help super-admin to view the data of admins
Actor	Admin – system
Pre-condition	Admin needed to see the data of all parking owner
Sequence of event	- Admin login - Admin click on mange parking owners - Table of parking owners appear
Post condition	Data of parking owner fetched
Exceptions	Lose connection with database

Table 8: Block parking owner

Use case name : Block parking owner	
Use case number	Uc-08
Use case description	The admin can delete the owner of the garage from the system
Actor	Admin – system
Pre-condition	All parking owners registered in our application.
Sequence of event	- Admin login to the system. - Admin will remove the owner of the garage from system
Post condition	System is ready to confirm new owner in application.
Exceptions	- There is no internet. - The owner of the garage is not involved in my application.

Table 9: Open admin dashboard

Use case name: Open admin dashboard	
Use case number	Uc-9
Use case description	The admin will have a dashboard that contains all the data about the system.
Actor	Admin – System
Pre-condition	- System be available. - System must be connected to the network.
Sequence of event	- Admin login to system. - Admin can see the details of my system. -The admin can see how many users are using my system.
Post condition	System is ready to show the higher user for my application
Exceptions	- There is no internet.

Table 10: Registration

Use case name: Registration	
Use case number	Uc-10
Use case description	Function enable customer to register for first time
Actor	User
Pre-condition	- System must be connected to the network. - User must be logged into the system. - User has valid e-mail
Sequence of event	- User open Application. - User click on create new account. - User: add his information (e-mail – username-password-mobile number – address). - User click Submit. - Account will create.
Post condition	User become has account
Exceptions	- User enter wrong data. - User not active the account

Table 11: Login

Use case name: Login	
Use case number	Uc-11
Use case description	- User login to system to access the functionality of the system. - The user's ability to change his/her password
Actor	User
Pre-condition	- User has account. - The site is available. - System must be connected to the network. - After a successful login a notification mail is sent to the user mail.
Sequence of event	- Enter username and password. - Validate username and password. - Allow access to system. - Invalid Username : - System shows an error message. - Invalid Password: - System shows an error message. - Invalid Password for 4 times: - Application closed.
Post condition	System is ready to find the nearest garage.
Exceptions	-The password is error. -The site not available. -The user not have email.

Table 12: Update user data

Use case name:	Update user data
Use case number	Uc-12
Use case description	User can Update his data and submit his new data.
Actor	User – System.
Pre-condition	- System be available. - There is internet, User logged in the site. - User update his data.
Sequence of event	- User Click on edit his registration. - User can edit his address. - User can submit his new data
Post condition	System display new data and save it in database.
Exceptions	- There is no internet. - There is error in login user

Table 13: Delete his account

Use case name: Delete his account	
Use case number	Uc-13
Use case description	User can delete his account from the application.
Actor	User – System
Pre-condition	- System be available. - System must be connected to the network.
Sequence of event	- User login to the system. - User will go to setting and go to his account. - User delete his account
Post condition	System is ready to create new account.
Exceptions	- There is no internet. - The application not available.

Table 14: Search for nearest parking

Use case name : Search for nearest parking	
Use case number	Uc-14
Use case description	Function Get the nearest parking in a specific area
Actor	User
Pre-condition	User has an account and logged in the application
Sequence of event	• User open app • User Log in with his account • User Click on find park • App would suggest and show the nearest park
Post condition	User got the nearest park

Table 15: show nearest parking

Use case name: show Nearest parking	
Use case number	Uc-15
Use case description	Function looks for free slot in nearest park
Actor	User
Pre-condition	User clicked on find and show the near parks
Sequence of event	- User after pressing find - The app show the near parks that have free slots
Post condition	User got the nearest park that has a free slot
Exceptions	All parks complete so there no free slots

Table 16: Make reservation

Use case name: Make reservation	
Use case number	Uc-16
Use case description	Function enable user to make a reservation
Actor	User
Pre-condition	The near park is chosen by customer has free slots
Sequence of event	• User after choosing the near park • Then click on reserve to reserve the slot
Post condition	User got reservation for slot in near park
Exceptions	The park is complete

Table 17: Make monthly or weekly reservation

Use case name: Make monthly or weekly reservation	
Use case number	Uc-17
Use case description	Function enable user to make a weekly or monthly reservation
Actor	User
Pre-condition	The near park is chosen by customer has free slots and allow long subscription.
Sequence of event	• User after choosing the near park • Then click on reserve and choose weekly or monthly reservation
Post condition	User got weekly or monthly reservation for slot in near park
Exceptions	The park is complete

Table 18: Scan QR code to enter

Use case name: Scan QR code to enter	
Use case number	Uc-18
Use case description	The case enable user to enter parking and park his car
Actor	User
Pre-condition	User has QR code of reservation details in this park
Sequence of event	- Customer scan QR code - If the QR code valid the barrier will open
Post condition	Barrier open and customer pass to his slot
Exceptions	No reservation

Table 19: Scan QR code to exit

Use case name : Scan QR code to exit	
Use case number	Uc-19
Use case description	The case enable user to pay and go out parking
Actor	User
Pre-condition	User already parked has a reservation and parked his car
Sequence of event	• User scan QR code • If the QR code valid the system display the bill to complete payment
Post condition	User has a bill
Exceptions	The QR code invalid

Table 20: Make payment

Use case name : Make payment	
Use case number	Uc-20
Use case description	System will automatically Withdraw the amount form user's visa
Actor	System
Pre-condition	user scan is QR code and get out from the parking
Sequence of event	• user scans his QR code to get out form the parking, after parking processing finish and he spent his time • System will determine how long time user spent • system will determine how much money he should pay depending on his time that he spent inter and price per hour in this parking. • System will automatically Withdraw the amount form customer's visa.
Post condition	• user can rate the service (it's optional) • System is ready for another user
Exceptions	• There is no internet • Customer doesn't have money on his visa

Table 21: Give rate

Use case name: Give rate	
Use case number	Uc-21
Use case description	Function enable customer to rate the service to improve it in future
Actor	user
Pre-condition	user scanned his QR code and get out from the parking and pay his payment
Sequence of event	• usercan rate the service from 1 to 5 stars according to his satisfaction (it's optional)
Post condition	System is ready for another customer.
Exceptions	- There is no internet - User doesn't want to rate the service

Table 22: Logout

Use case name: Logout	
Use case number	Uc-22
Use case description	Function enable customer to Log out from the App at any time.
Actor	user
Pre-condition	user register or logged into the App
Sequence of event	User click on log out button
Post condition	System is ready for login from new user
Exceptions	- There is no internet - User didn't login or doesn't have account

Table 23: User authentication

Use case name : User authentication	
Use case number	Uc-23
Use case description	when user login system will check his data as username, email and password and compare them with his data which saved in database from his registration, if they are typical customer can login if not customer can create new account.
Actor	System
Pre-condition	user logged into the app before
Sequence of event	• user register to the app in his first time • System will save customer data as username, email and password • user request to login to the app in the second time and enter his data • System will compare user data with his data which saved in data base • if they are typical system will allow user to enter the app • if data not typical user can create new account
Post condition	System is ready for another user
Exceptions	• There is no internet • user's data are not typical

Table 24: Generate QR code for reservation

Use case name: Generate QR code for reservation	
Use case number	Uc-24
Use case description	System will generate QR code to allow user to enter the parking by scanning it.
Actor	System
Pre-condition	user clicks on reservation button from the app
Sequence of event	• user clicks on reservation button on the app • App will generate QR code for user to scan it to enter and get out the parking and this QR code include reservation details.
Post condition	• user scans his QR code to enter to the parking.
Exceptions	• There is no internet • user doesn't click on reservation button

Table 25: Calculate the cost of parking

Use case name : Calculate the cost of parking	
Use case number	Uc-25
Use case description	System will calculate the amount of money that user should pay for parking according to time that he spent enter the parking
Actor	System
Pre-condition	user scan is QR code and get out from the park after parking
Sequence of event	• processing finish and user spent his time • System will determine how long time user spent • System will determine how much money he should pay depending on his time that
Post condition	System will automatically Withdraw the amount form user's visa
Exceptions	There is no internet

Table 26: make payment

Use case name: make payment	
Use case number	Uc-26
Use case description	when user make payment at leaving the parking
Actor	User – System
Pre-condition	Visa Information – Internet
Sequence of event	• System shows the balance • User enter his visa information • System checks his account and the balance • System deducts a price from his account • The use case ends.
Post condition	System ready for new process
Exceptions	Account has not enough balance to done reservation process – Error in a system – Error in visa information – Internet disconnected Network – Network

Table 27: Registration

Use case name : Registration	
Use case number	Uc-27
Use case description	A parking owner creates an account for his parking
Actor	Parking owner – System
Pre-condition	Parking owner not member in our system
Sequence of event	•The System prompts the user for a username and password or register new account. •The user selects registration option. •The System prompts user for registration information, Username, password, etc. •The user enters in their information. •System verifies information and creates account. •The use case ends.
Post condition	•System ready for new registration process
Exceptions	Invalid information than user enter – Error in a system – Internet disconnected Network

Table 28: Login

Use case name: Login	
Use case number	Uc-28
Use case description	A parking owner logs in to the System
Actor	Parking owner – system
Pre-condition	•The System prompts the user for a username and password or register new account •The user enters his/her username and password. •The system validates the entered username and password, making sure that the entered username is a valid username in the System, and that the required password is entered for the entered username. •The user is signed in and returned to the home page as a Logged In User. •The use case ends.
Sequence of event	System ready for new login process
Post condition	System ready for new login process
Exceptions	Invalid username or password – Network problems – Internet connections problems

Table 29: Update info

Use case name: Update info	
Use case number	Uc-29
Use case description	Parking owner can update his parking information.
Actor	Parking owner – System – Data Base
Pre-condition	Internet connection - User has an account already
Sequence of event	• Parking owner login • Parking owner update this parking information in our database • Set saves this updates • The use case ends.
Post condition	System ready for new process
Exceptions	Enter invalid information - Network problems – Internet connections problems.

Table 30: Delete

Use case name :Delete Account	
Use case number	Uc-30
Use case description	Parking owner delete his/her account.
Actor	Parking owner - system
Pre-condition	User has an account already – Internet connection.
Sequence of event	• User login his/her account • System verifies information and login. • User choose delete an account option. • System shows message alert to confirm this instruction. • User confirm this instruction • An account was deleted • The use case ends.
Post condition	System ready for any Instructions or processes
Exceptions	User does not confirm this instruction – Internet connection problems – Network problems – invalid password or username to login

Table 31: Open parking dashboard

Use case name : Open parking dashboard	
Use case number	Uc-31
Use case description	Show dashboard (data for each parking or garage).
Actor	Parking owner - System
Pre-condition	Parking owner has an account already – Internet connection
Sequence of event	• Parking owner login into his account • System verifies information and login. • Parking owner choose dashboard option. • System opens dashboard page • Parking owner shows his dashboard and can print it . • The use case ends
Post condition	System ready for any Instructions or processes
Exceptions	Internet connection problems – Network problems – invalid password or username to login

Table 32: Logout

Use case name	Logout
Use case number	Uc-32
Use case description	Function enable parking owner to Logout from system
Actor	Parking owner system
Pre-condition	Parking owner logged into the App
Sequence of event	The user click on logout button
Post condition	System is ready for login from new user
Exceptions	There is no internet User didn't login or doesn't have account

Table 33: Receive reservation request

Use case name:	Receive reservation request
Use case number	Uc-33
Use case description	Function send the request of reservation
Actor	System - user
Pre-condition	users make reservation
Sequence of event	- user after making reservation - Then click on reserve to reserve the reservation
Post condition	user make reservation in near park (system confirm the reservation)
Exceptions	- Error in reservation - The park is complete

Table 34: Confirm reservation

Use case name :	Confirm reservation
Use case number	Uc-34
Use case description	Function confirm a reservation by the system
Actor	System - user
Pre-condition	User send reservation request and wait for response
Sequence of event	- user request for reservation
Post condition	users receive reservation confirm to nearest park
Exceptions	- Error in reservation - The park is complete

Table 35: Open baffle

Use case name:	
Open baffle	
Use case number	Uc-35
Use case description	There are baffles on every car area to prevent any user tries to park in one other his one
Actor	System - user
Pre-condition	User must be scan QR code
Sequence of event	1. User drive into the check-in counter. 2. User sign in by scanning his/her code to identify. 3. The system finds the parking slot for a user. 4. The system shows the assigned parking slot data on the screen. 5. The navigation system displays the route to the parking slot for the customer to follow.
Post condition	Users make confirm reservation and baffle opened
Exceptions	1. User is not registered yet - system disallows him/her to park; see use case Register. 2. User wants to log in the mobile application. see use case Login Application 3. No parking slot available.

Table 36: Close baffle

Use case name: Close baffle	
Use case number	Uc-36
Use case description	Baffle will closed after user leave the area
Actor	System - user
Pre-condition	User scan QR code to leave the area
Sequence of event	1. User drive out from the parking area into the check-out counter. 2. User sign out by scanning his/her code to identify. 3. The system allows the user to leave. 4. The system updates the parking slot status to empty
Post condition	Baffle closed
Exceptions	1. User is not registered yet - system disallows him/her to leave. 2. user is not the same person as when check-in; system disallows him/her to leave; the security guard finds out who is the drivers; see use case Register, Security.

Table 37: Change value of parking area

Use case name: Change value of parking area	
Use case number	Uc-37
Use case description	Function that change the value of parking area to know the parking area is empty or not
Actor	System
Pre-condition	Parking area take value 0 or 1
Sequence of event	- when user enter the parking and scan QR code - This value change to 1 that mean that is not empty - When user scan QR code to leave the parking - This value change from 1 to 0 that mean this is an empty slot.
Post condition	The value changed
Exceptions	Connection lost between hardware and database

Chapter (4) Deployment & layout

Welcome to the chapter on implementing and deploying our project! In this chapter, we will discuss the practical aspects of bringing our project to life and deploying it for actual use. We will explore the implementation process and provide insights into project layout, ensuring a successful realization of our project's goals.

Implementation Processing this section, we will delve into the implementation process, where we transform our design and requirements into functional software. We will explore the tools, programming languages, and frameworks utilized in the development of our project. We will discuss the step-by-step approach to coding, testing, and debugging, ensuring a high-quality implementation. Furthermore, we will address any challenges or considerations specific to our project's technology stack.

Project Layout In this section, we will present the project layout, providing a structured overview of the project's components and their organization. We will discuss the directory structure, naming conventions, and file organization that ensure clarity and maintainability. The project layout enables a logical arrangement of code, resources, and documentation, facilitating collaboration and ease of maintenance for both individual developers and teams.

Deployment In this section, we will explore the deployment of our project to make it accessible to end-users. We will discuss the various deployment options available, such as local deployment, cloud hosting, or web-based deployment. We will cover topics like server configuration, database setup, and network considerations to ensure a seamless deployment process. Additionally, we will address security concerns and best practices to protect our deployed project from potential threats.

Throughout this chapter, we will provide practical examples and guidelines to help you navigate the implementation and deployment of your project. By the end of this chapter, you will have a clear understanding of the implementation

process, project layout, and deployment considerations necessary to bring your project to fruition. Let's dive into the exciting world of implementing and deploying our project!

Mobile App “Rakna”

Rakna is a cross-platform mobile application developed using React Native and Laravel, designed to revolutionize the way users make reservations. With its seamless integration of front-end and back-end technologies, Rakna provides a user-friendly platform for individuals to effortlessly book parking area.

Mobile App layout

1-Onboarding screen



Figure 9: App Onboarding screen

2-Get started screen

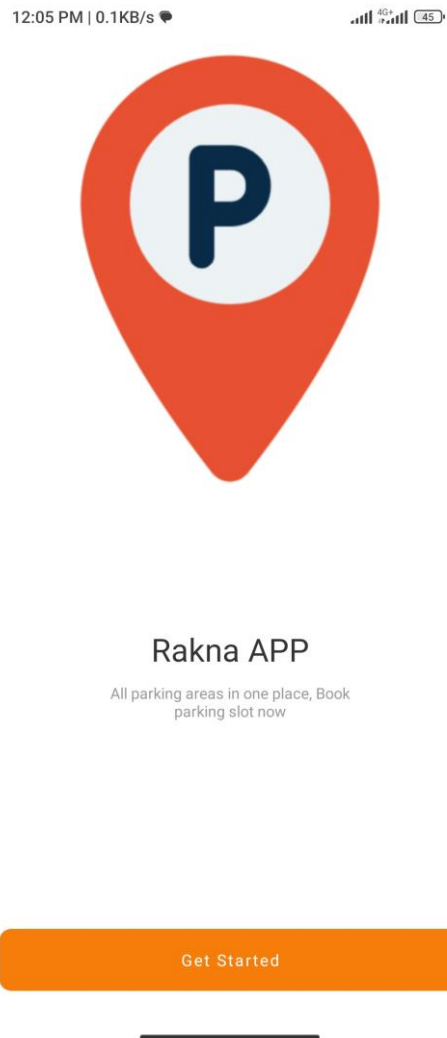


Figure 10: App Get started screen

3-Sign in screen

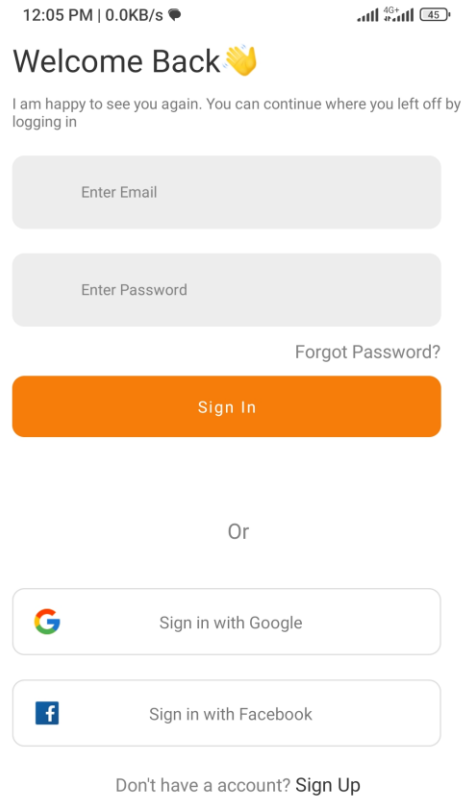
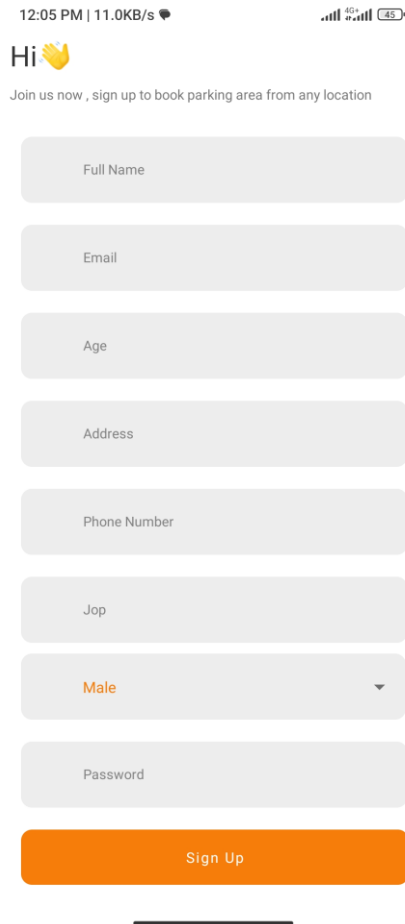


Figure 11: App Sign in screen

A login screen in a React Native app that allows users to enter their email and password, login using the provided credentials, and navigate to the home screen upon successful login. It also includes UI elements for social login options and a link to the sign-up screen.

4-sign up



12:05 PM | 11.0KB/s | 4G+ | 45%

Hi 🖐️

Join us now , sign up to book parking area from any location

Full Name

Email

Age

Address

Phone Number

Jop

Male ▼

Password

Sign Up

Figure 12: App sign up

These components provide the UI and functionality for user registration screens in a React Native application by take some state variables (**name**, **email**, **age**, **address**, **phone**, **job**, **gender**, and **password**) are used to store user registration data.

5-Home screen

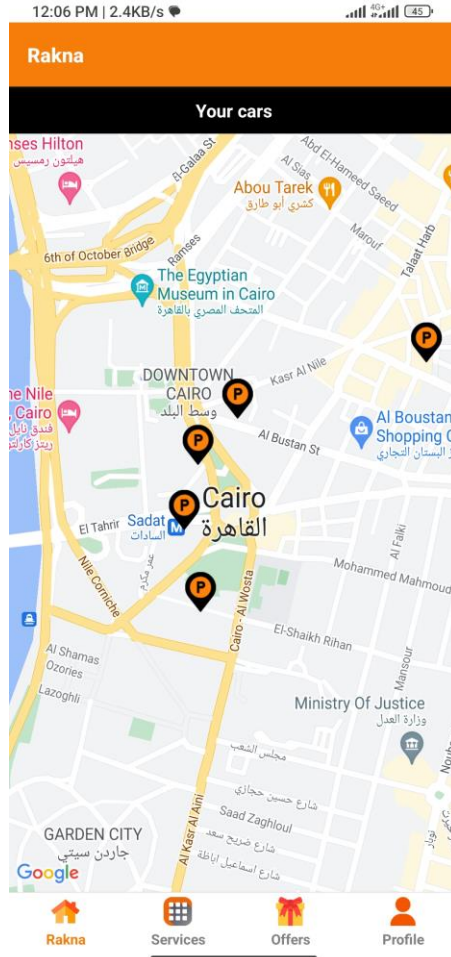


Figure 13: App Home screen

This screen sets up a bottom tab navigation with four tabs: "Rakna" (Map), "Services", "Offers", and "Profile". Each tab is associated with a specific component, and icons are displayed for each tab, main screen provide data of locations of parking areas via google maps API. When press on each icon provide all information about this parking area.

6-parking details

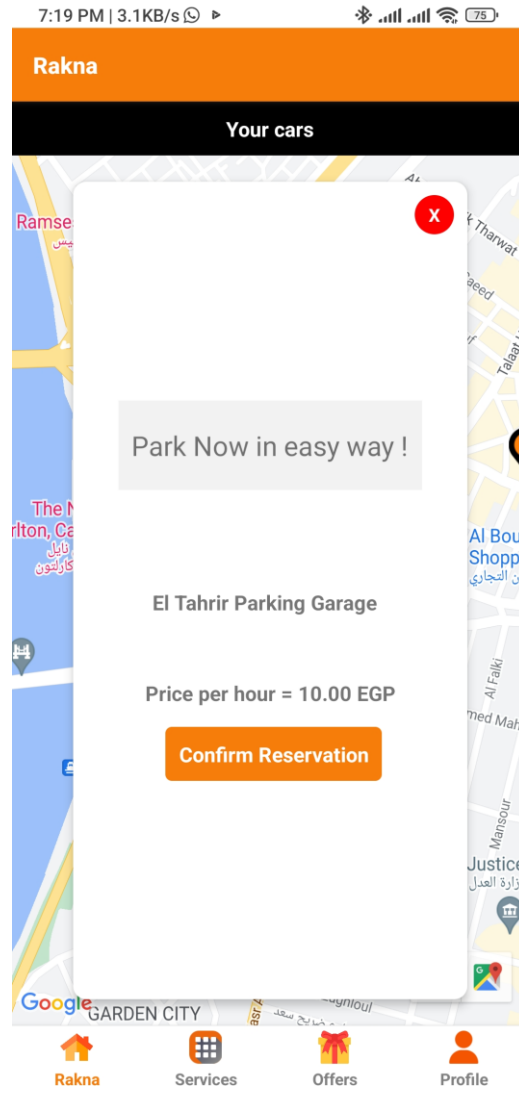


Figure 14: App parking details pop up

When press on certain icon it will appear the details of this location like name and price per hour.

7- Reservation screen

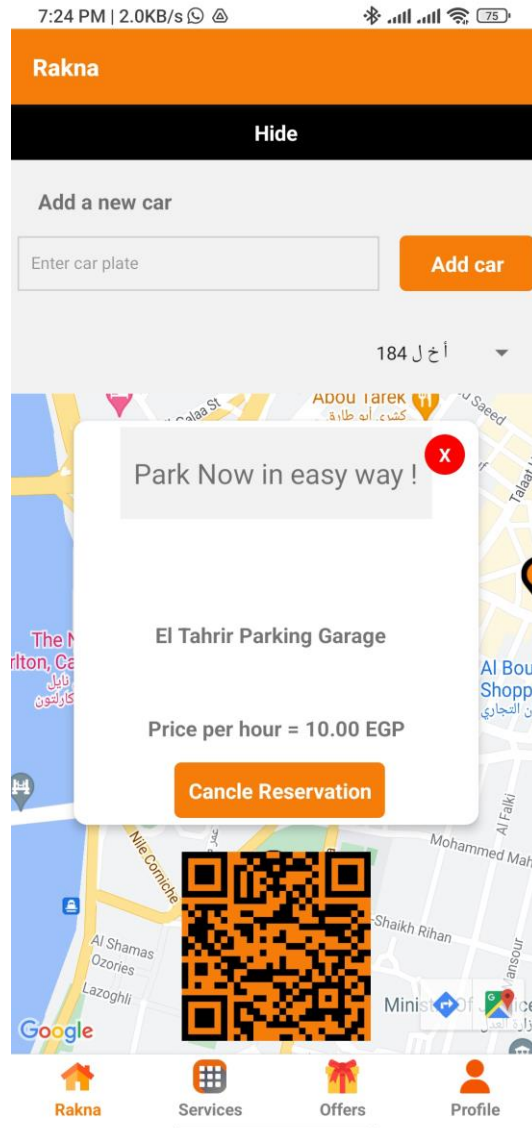


Figure 15: APP Reservation screen

A reservation screen allow user to make reservation for one of parking then QR code will appear carry all information about reservation also can cancel this reservation

8-Extra Screens

a- Profile screen

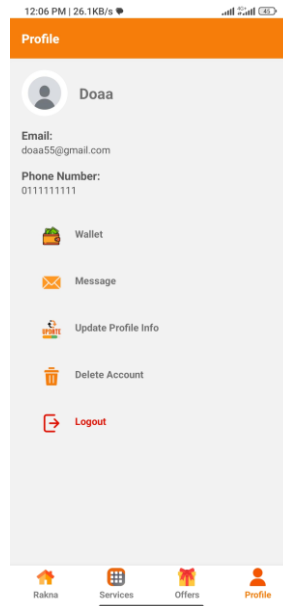


Figure 16: App Profile screen

b- Services screen

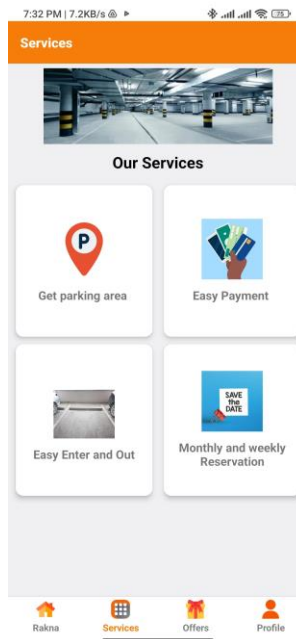


Figure 17: App services screen

c- Offers screen

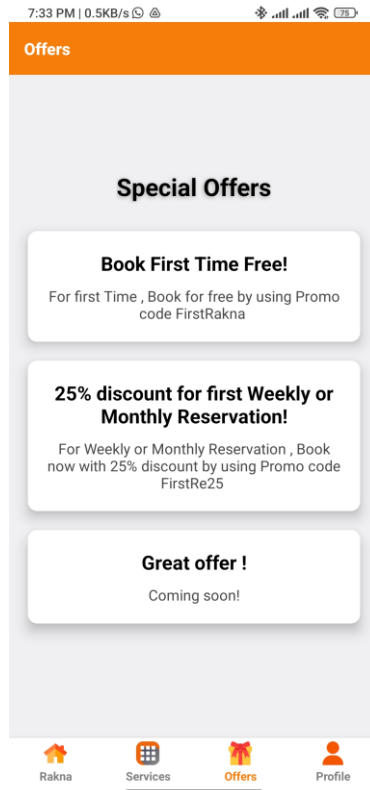


Figure 18:App offers screen

Web App

Web application built with JavaScript and Laravel, designed specifically for parking owners. This powerful platform allows parking owners to effortlessly sign in and access a comprehensive dashboard, providing complete control over their parking data. With features such as intuitive charts and graphs, the application enables parking owners to gain valuable insights into their income, helping them make informed decisions to optimize their business operations. Additionally, the web app includes an admin panel, granting administrators the ability to efficiently manage the entire system. From overseeing parking spaces to monitoring transactions, our application offers a seamless and efficient solution for parking management. Say goodbye to complex manual processes and embrace the convenience and effectiveness of our JS and Laravel-powered web app for comprehensive parking management.

Web layout

Login:

To manage your garage account, you can log in by adding your email and password

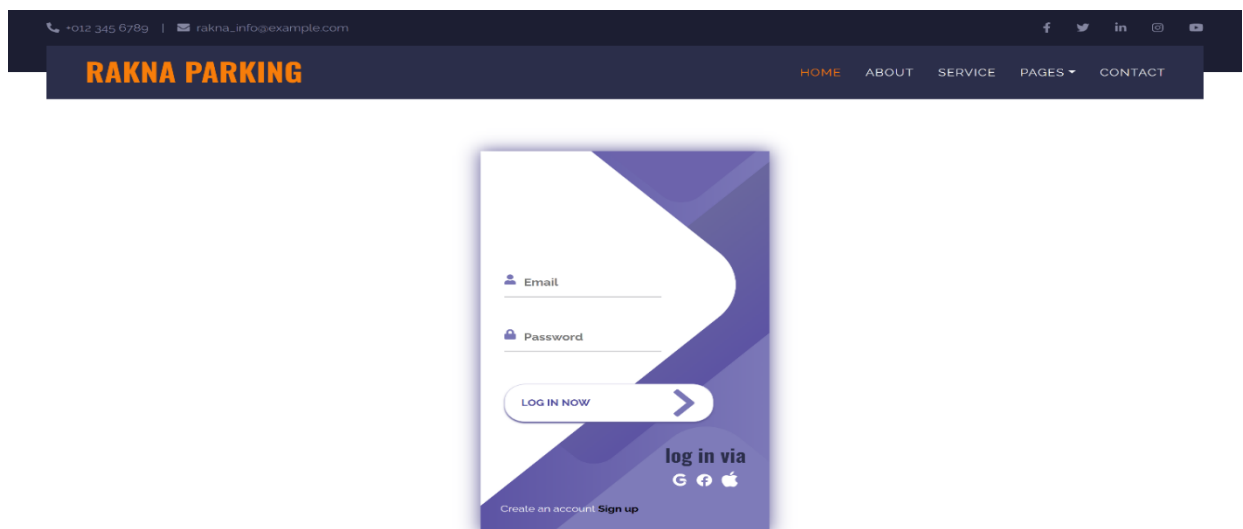


Figure 19: Web login

Registration:

The registration web page allows users to create an account by providing their garage and personal information, such as name, email address, garage location, number of floors, and password.

RAKNA PARKING [HOME](#) [ABOUT](#) [SERVICE](#) [PAGES](#) [CONTACT](#)

Full Name

Username

Your Email

Password

Confirm Password

Phone Number

Birth Date - dd/mm/yy

Job

Parking Address

Gender Select Your Gender

[REGISTRATION NOW](#)

log in via

[G](#) [f](#) [A](#)

Already a member [Log in](#)

Figure 20: Web Registration

User Dashboard:

The dashboard allows our client (Garage Owner) to monitor their garage, track daily and overall profits, and obtain information about the current garage status.

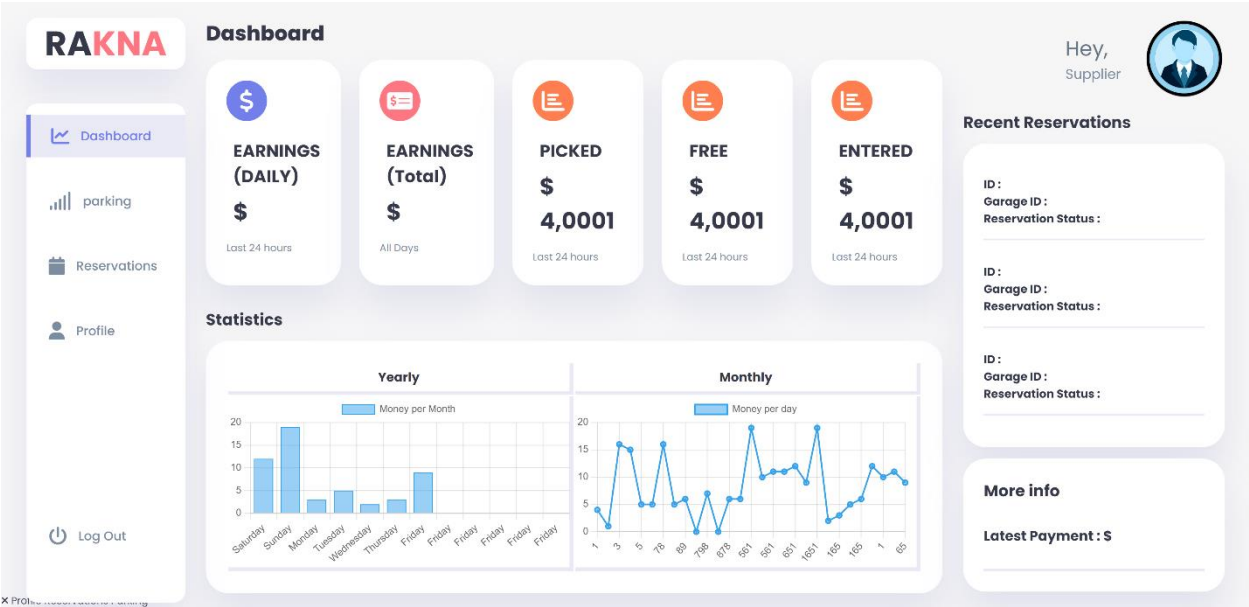


Figure 21: Web User Dashboard

Admin Dashboard:

"We can use it to know everything related to the site, customers, and the garages we deal with. Additionally, we can manage all matters related to the system, like add or delete."

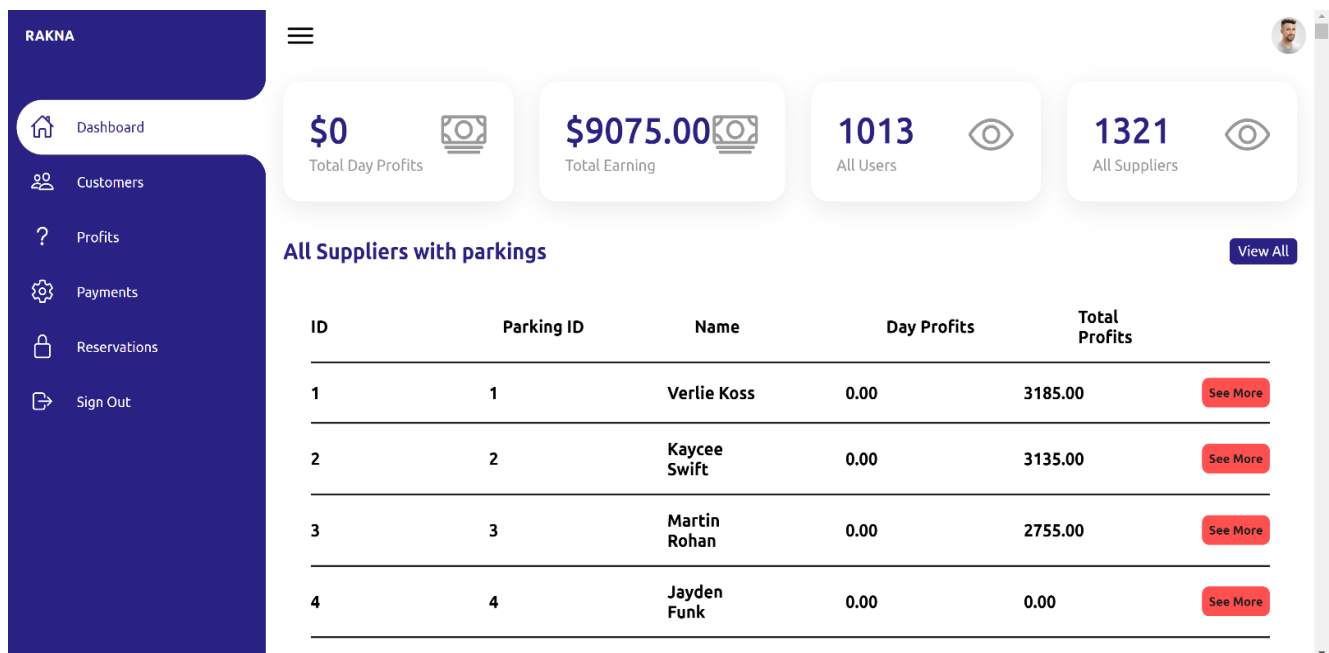


Figure 22: Web Admin Dashboard

Hardware

Analysis

We will evaluate the performance of the implemented embedded system. The performance evaluation aimed to determine how well the system meets the project objectives and requirements. In this chapter, we describe the performance evaluation methods used, including benchmarking and testing. The results of the performance evaluation are presented, discussed, and compared with existing literature and industry standards. We also identify any limitations of the performance evaluation and provide recommendations for future work.

Aim:

The aim of this application is to design such project that calculate number of car parking and know slots full or empty.

This project comprises microcontroller one is for decision making. IR sensor is used to detect specific characteristics in its surroundings through emitting. If IR enter sensor detect car, motor turn on to open gate. If IR back sensor detect car, motor turn off to close gate. Another IR sensors is sending signal to processor to physical signals into digital data if they detect car to know slot is empty or full.

Components:

1. ARDUINO UNO

Arduino UNO is a microcontroller board based on the ATmega328P. It has 14 digital input/output pins (of which 6 can be used as PWM outputs), 6 analog inputs, a 16 MHz ceramic resonator, a USB connection, a power jack, an ICSP header and a reset button. It contains everything needed to support the microcontroller; simply connect it to a computer with a USB cable or power it with a AC-to-DC adapter or battery to get started. You can tinker with your UNO without worrying too much about doing something wrong, worst-case scenario you can replace the chip for a few dollars and start over again.

2. LCD

A liquid-crystal display (LCD) is a flat-panel display or other electronically modulated optical device that uses the light-modulating properties of liquid crystals combined with polarizers. Liquid crystals do not emit light directly but instead use a backlight or reflector to produce images in color or monochrome. In LCD two separate routines are used for two separate cycles. One is used for transferring the data or character and another is used for sending the command inside an embedded system. For preventing the crash of data, we must allow the LCD to complete the execution of each operation.

3. LED

A light-emitting diode (LED) is a semiconductor device that emits light when an electric current flows through it. When current passes through an LED, the electrons recombine with holes emitting light in the process. LEDs allow the current to flow in the forward direction and blocks the current in the reverse direction.

4. MOTOR

any of a class of rotary electrical motors that converts direct current (DC) electrical energy into mechanical energy. The most common types rely on the forces produced by magnetic fields. Nearly all types of DC motors have some internal mechanism, either electromechanical or electronic, to periodically change the direction of current in part of the motor.

5. Sensor

A sensor is a device that detects and responds to some type of input from the physical environment. The input can be light, heat, motion, moisture or pressure and any number of other.

Tools:

1-The Arduino Integrated Development Environment - or Arduino Software (IDE) - contains a text editor for writing code, a message area, a text console, a toolbar with buttons for common functions and a series of menus. It connects to the Arduino hardware to upload programs and communicate with them.

2-The Proteus Design Suite is a proprietary software tool suite used primarily for electronic design automation. The software is used mainly by electronic design engineers and technicians to create schematics and electronic prints for manufacturing printed circuit boards. Lab center Electronics Ltd.

Design

We will present the process of integrating the hardware and software components of an embedded system. In this chapter, we describe how we integrated the components and any challenges we faced during the integration process. We also explain how we validated the design and tested the integrated system to ensure that it meets the project objectives and requirements. Finally, we discuss any design optimizations made to improve the system's performance, cost, or other relevant factors.

Arduino Architecture:

Arduino's processor basically uses the Harvard architecture where the program code and program data have separate memory. It consists of two memories- Program memory and the data memory. The code is stored in the flash program memory, whereas the data is stored in the data memory. The Atmega328 has 32 KB of flash memory for storing code (of which 0.5 KB is used for the bootloader), 2 KB of SRAM and 1 KB of EEPROM and operates with a clock speed of 16MHz.

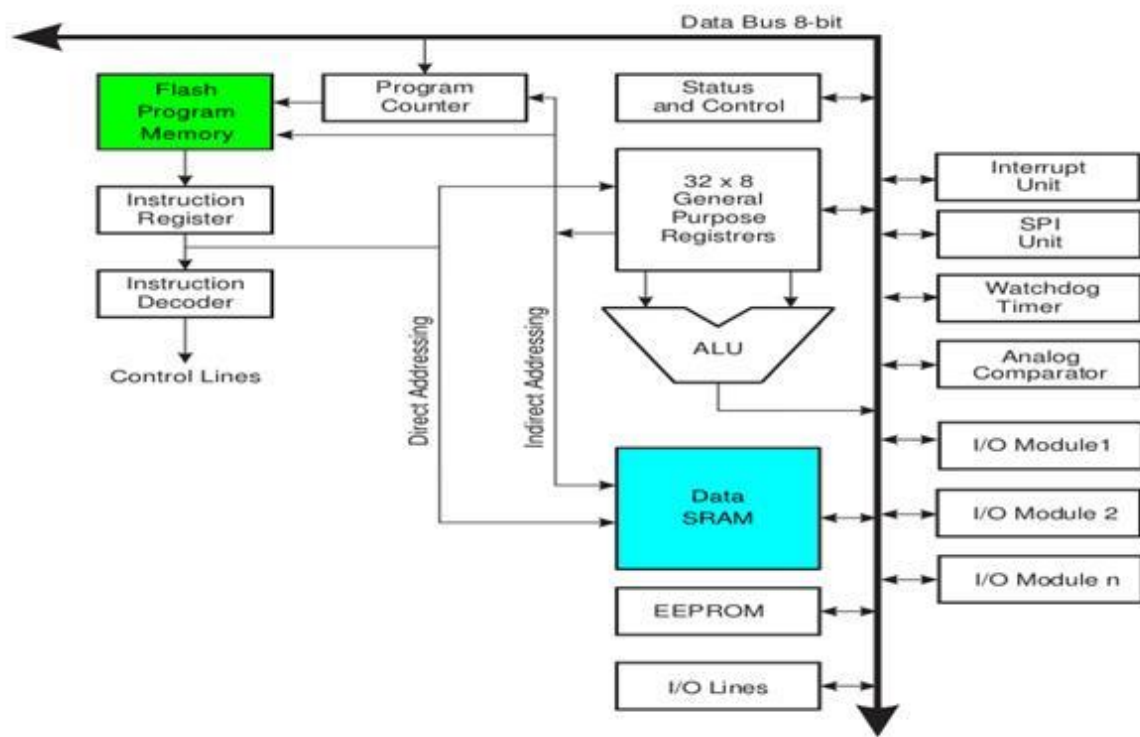


Figure 23: Arduino Architecture

Arduino Pin Diagram:

Arduino Uno consists of 14 digital input/output pins (of which 6 can be used as PWM outputs), 6 analog inputs, a 16 MHz crystal oscillator, a USB connection, a power jack, an ICSP header, and a reset button.

Power Jack: Arduino can be power either from the pc through a USB or through external source like adaptor or a battery. It can operate on a external supply of 7 to 12V. Power can be applied externally through the pin Vin or by giving voltage reference through the IOREf pin.

Digital Inputs: It consists of 14 digital inputs/output pins, each of which provide or take up 40mA current. Some of them have special functions like pins 0 and 1, which act as Rx and Tx respectively, for serial communication, pins 2 and 3- which are external interrupts, pins 3,5,6,9,11 which provides pwm output and pin 13 where LED is connected.

Analog inputs: It has 6 analog input/output pins, each providing a resolution of 10 bits.

AREf: It provides reference to the analog inputs.

Reset: It resets the microcontroller when low.

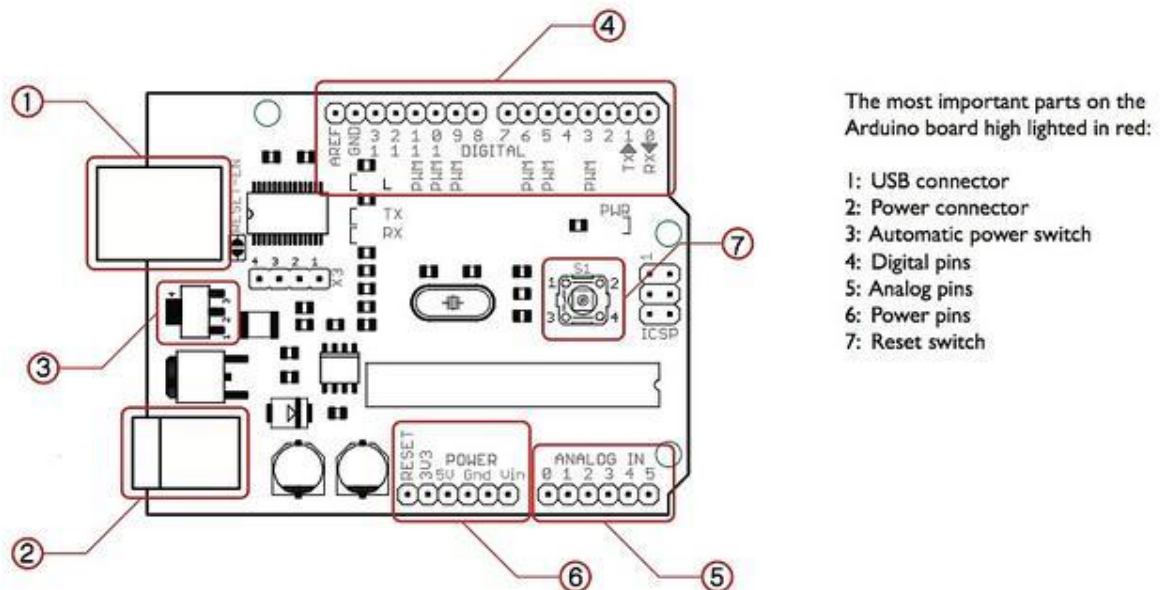


Figure 24: Arduino Pin Diagram

Design:

The initial phase design:

It is made to calculate the number of slots only.

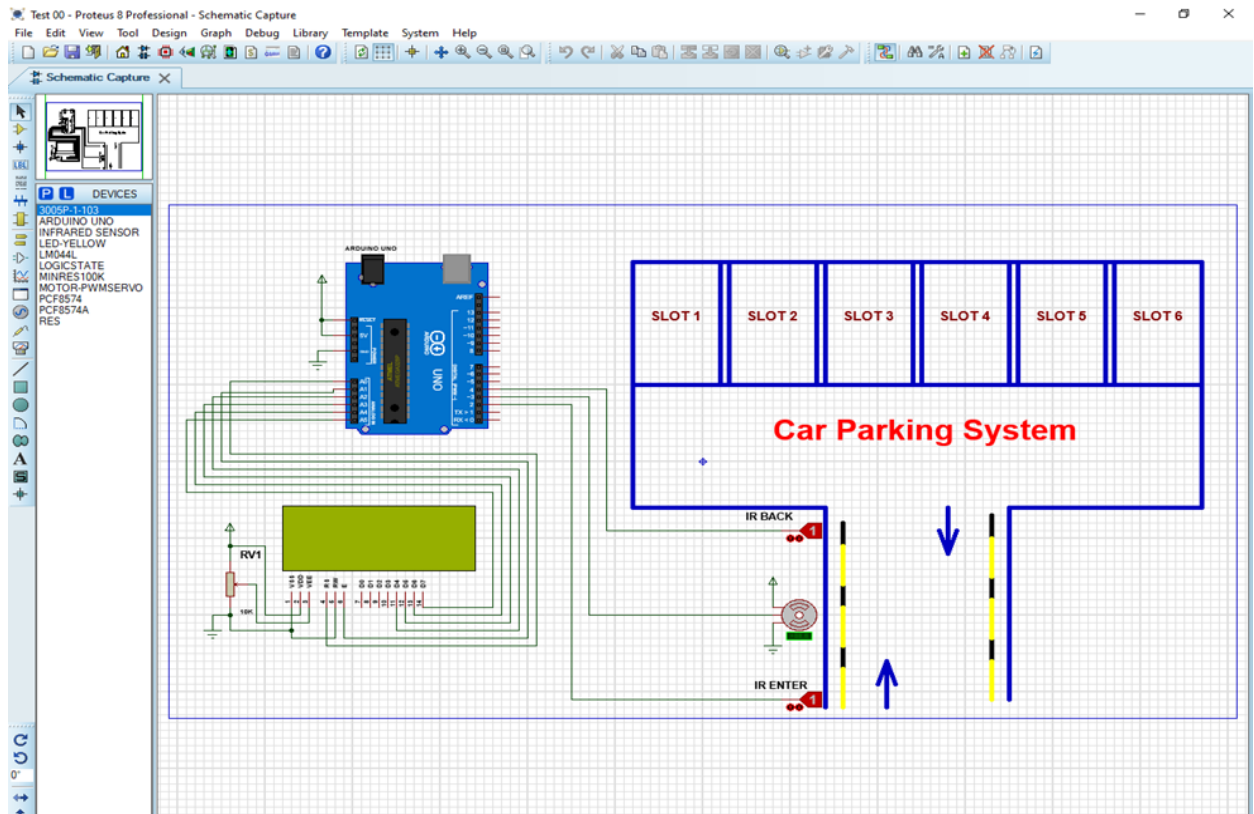


Figure 25: Hardware initial phase design

Implementation:

96

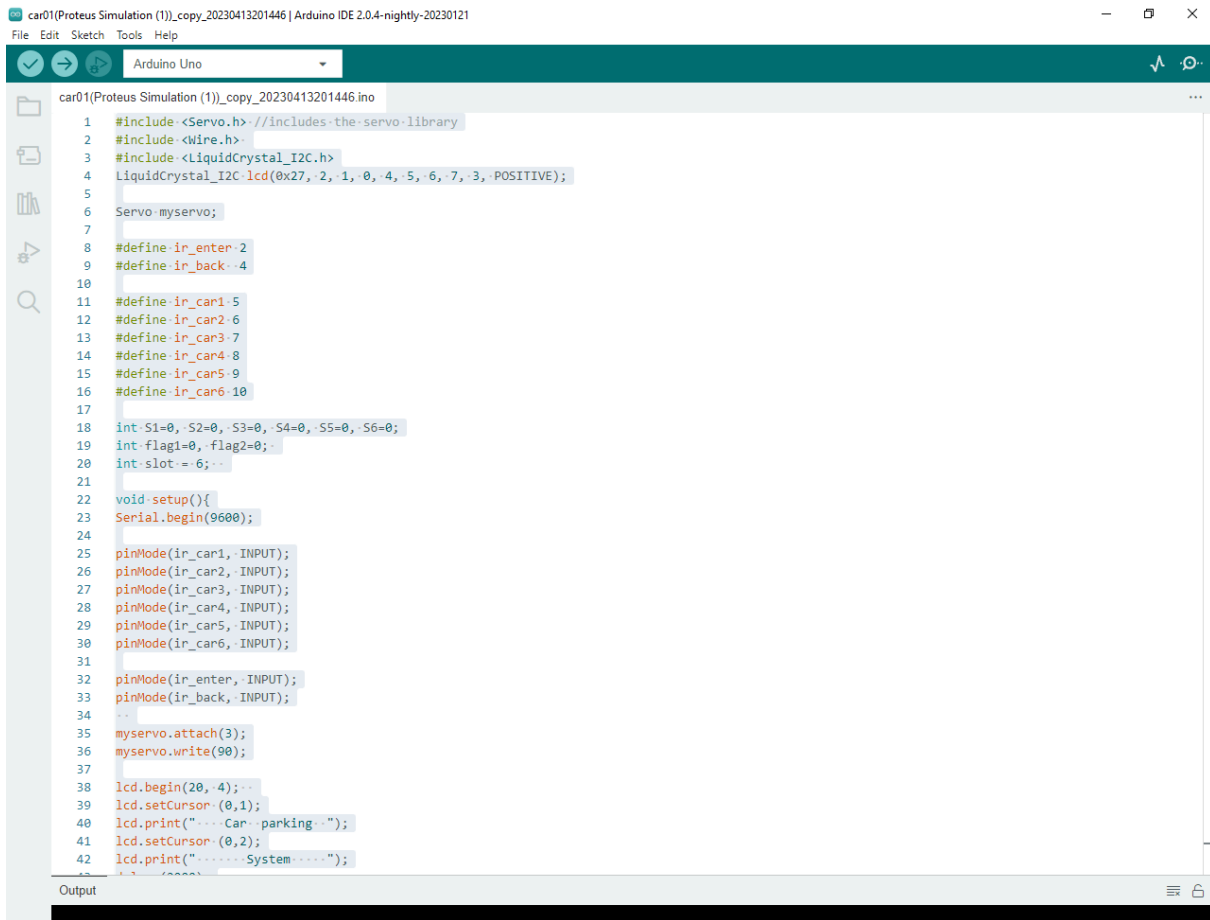


Figure 27: Arduino Uno 1

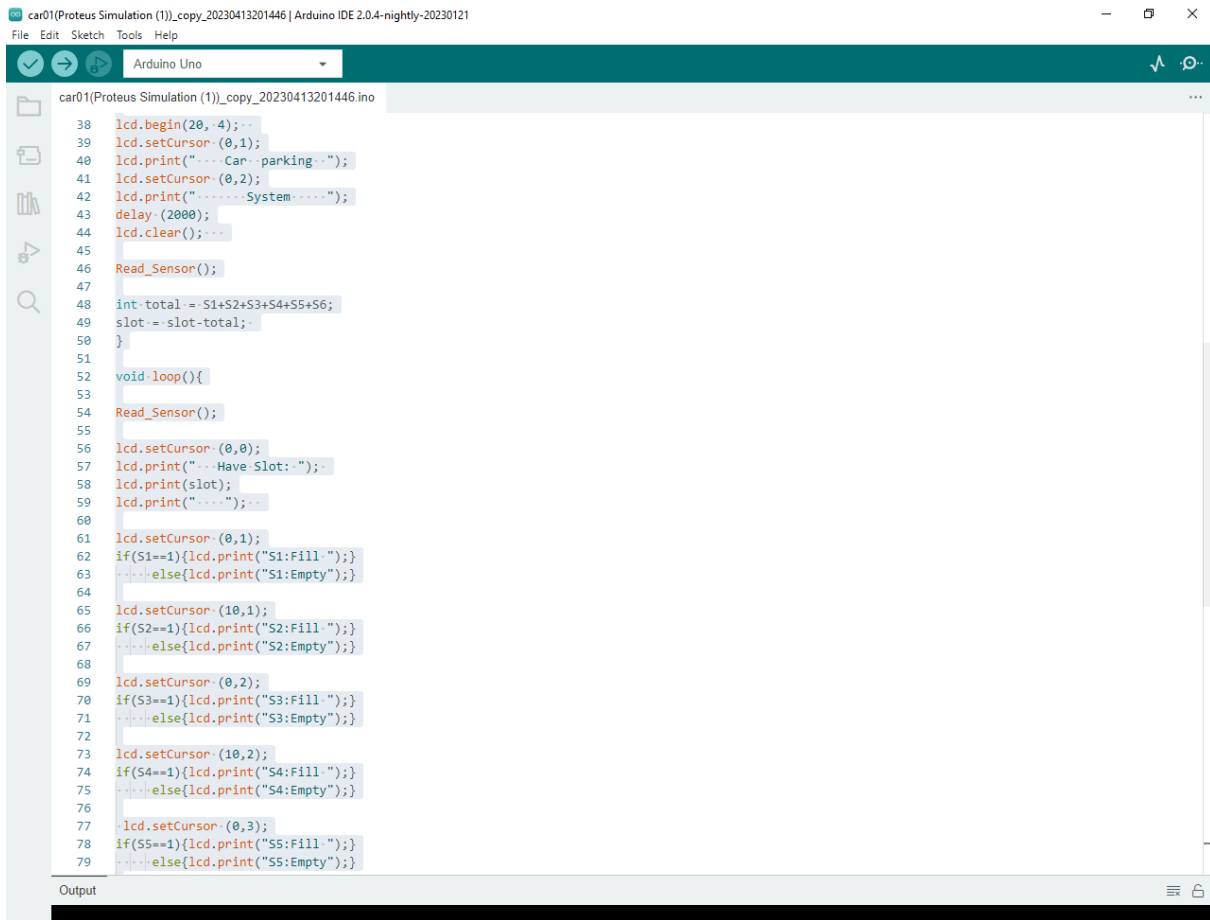


Figure 28: Arduino Uno 2

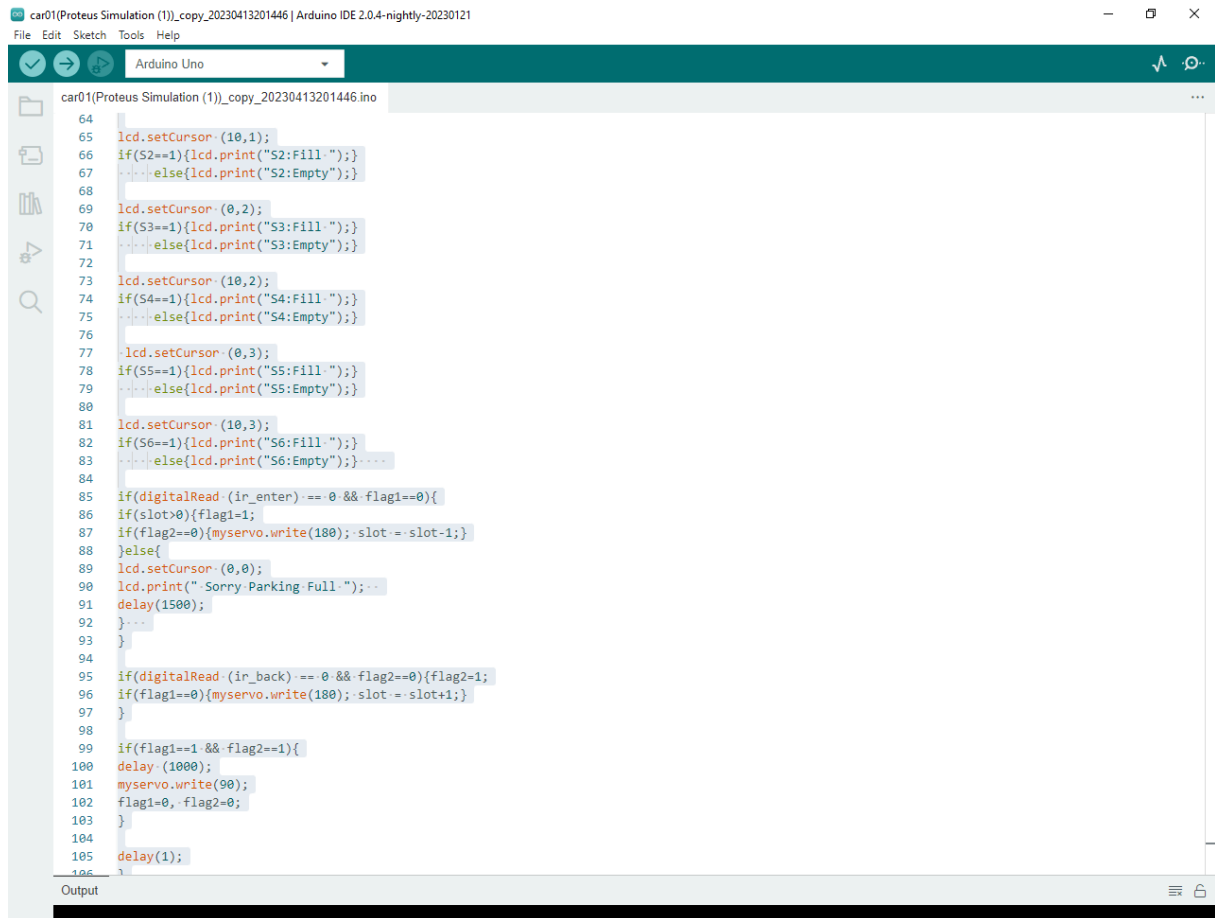


Figure 29: Arduino Uno 3

```
car01(Proteus Simulation (1))_copy_20230413201446 | Arduino IDE 2.0.4-nightly-20230121
File Edit Sketch Tools Help

Arduino Uno

car01(Proteus Simulation (1))_copy_20230413201446.ino
...
81 lcd.setCursor(10,3);
82 if(S6==1){lcd.print("S6:Fill.");}
83 else{lcd.print("S6:Empty");}...
84
85 if(digitalRead(ir_enter)==0 && flag1==0){
86 if(slot>0){flag1=1;
87 if(flag2==0){myservo.write(180); slot=slot-1;}
88 }else{
89 lcd.setCursor(0,0);
90 lcd.print(" Sorry Parking Full.");...
91 delay(1500);
92 }...
93 }
94
95 if(digitalRead(ir_back)==0 && flag2==0){flag2=1;
96 if(flag1==0){myservo.write(180); slot=slot+1;}
97 }
98
99 if(flag1==1 && flag2==1){
100 delay(1000);
101 myservo.write(90);
102 flag1=0, flag2=0;
103 }
104
105 delay(1);
106 }
107
108 void Read_Sensor(){
109 S1=0, S2=0, S3=0, S4=0, S5=0, S6=0;
110
111 if(digitalRead(ir_car1)==0){S1=1;}
112 if(digitalRead(ir_car2)==0){S2=1;}
113 if(digitalRead(ir_car3)==0){S3=1;}
114 if(digitalRead(ir_car4)==0){S4=1;}
115 if(digitalRead(ir_car5)==0){S5=1;}
116 if(digitalRead(ir_car6)==0){S6=1;}...
117 }
118
119
120
121
122

Output
```

Figure 30: : Arduino Uno 4

```
car01(Proteus Simulation (1))_copy_20230413201446 | Arduino IDE 2.0.4-nightly-20230121
File Edit Sketch Tools Help

Arduino Uno

car01(Proteus Simulation (1))_copy_20230413201446.ino
...
81 lcd.setCursor(10,3);
82 if(S6==1){lcd.print("S6:Fill.");}
83 else{lcd.print("S6:Empty");}...
84
85 if(digitalRead(ir_enter)==0 && flag1==0){
86 if(slot>0){flag1=1;
87 if(flag2==0){myservo.write(180); slot=slot-1;}
88 }else{
89 lcd.setCursor(0,0);
90 lcd.print(" Sorry Parking Full.");...
91 delay(1500);
92 }...
93 }
94
95 if(digitalRead(ir_back)==0 && flag2==0){flag2=1;
96 if(flag1==0){myservo.write(180); slot=slot+1;}
97 }
98
99 if(flag1==1 && flag2==1){
100 delay(1000);
101 myservo.write(90);
102 flag1=0, flag2=0;
103 }
104
105 delay(1);
106 }
107
108 void Read_Sensor(){
109 S1=0, S2=0, S3=0, S4=0, S5=0, S6=0;
110
111 if(digitalRead(ir_car1)==0){S1=1;}
112 if(digitalRead(ir_car2)==0){S2=1;}
113 if(digitalRead(ir_car3)==0){S3=1;}
114 if(digitalRead(ir_car4)==0){S4=1;}
115 if(digitalRead(ir_car5)==0){S5=1;}
116 if(digitalRead(ir_car6)==0){S6=1;}...
117 }
118
119
120
121
122

Output
```

Figure 31: Arduino Uno 5

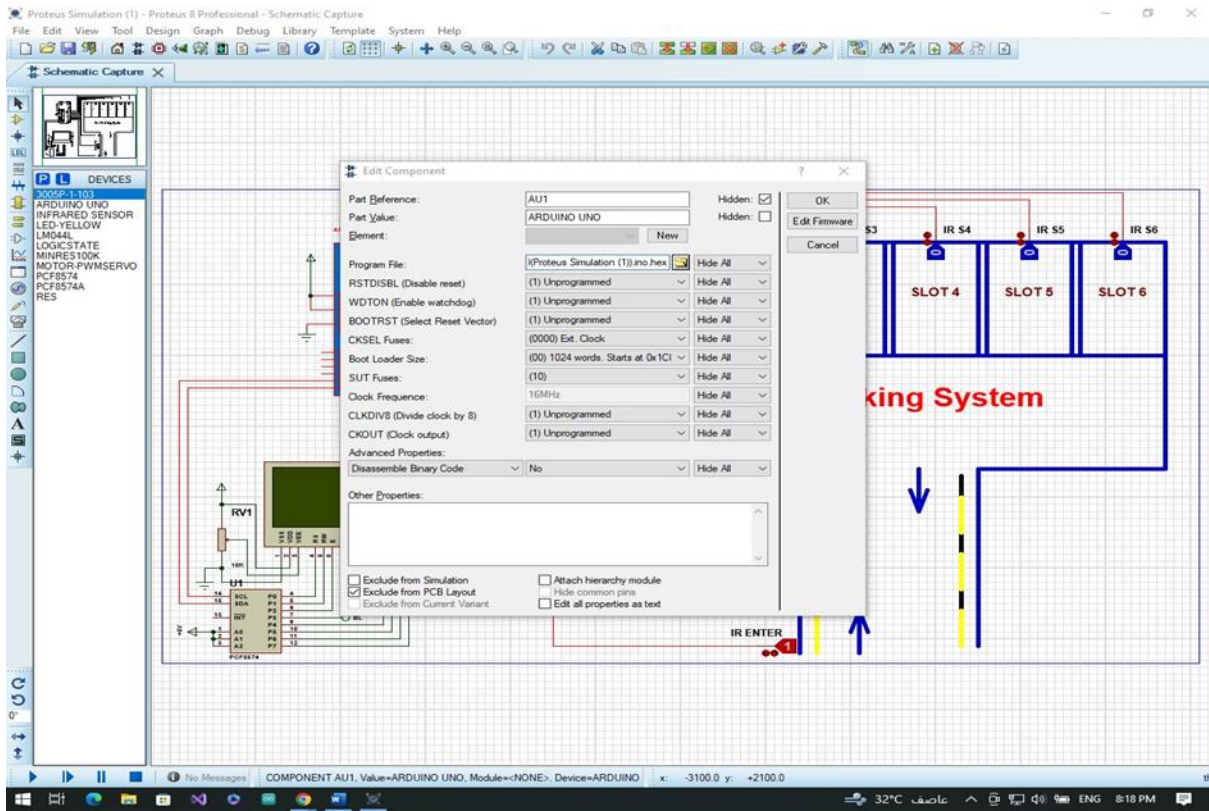


Figure 32: Hardware Implementation 1

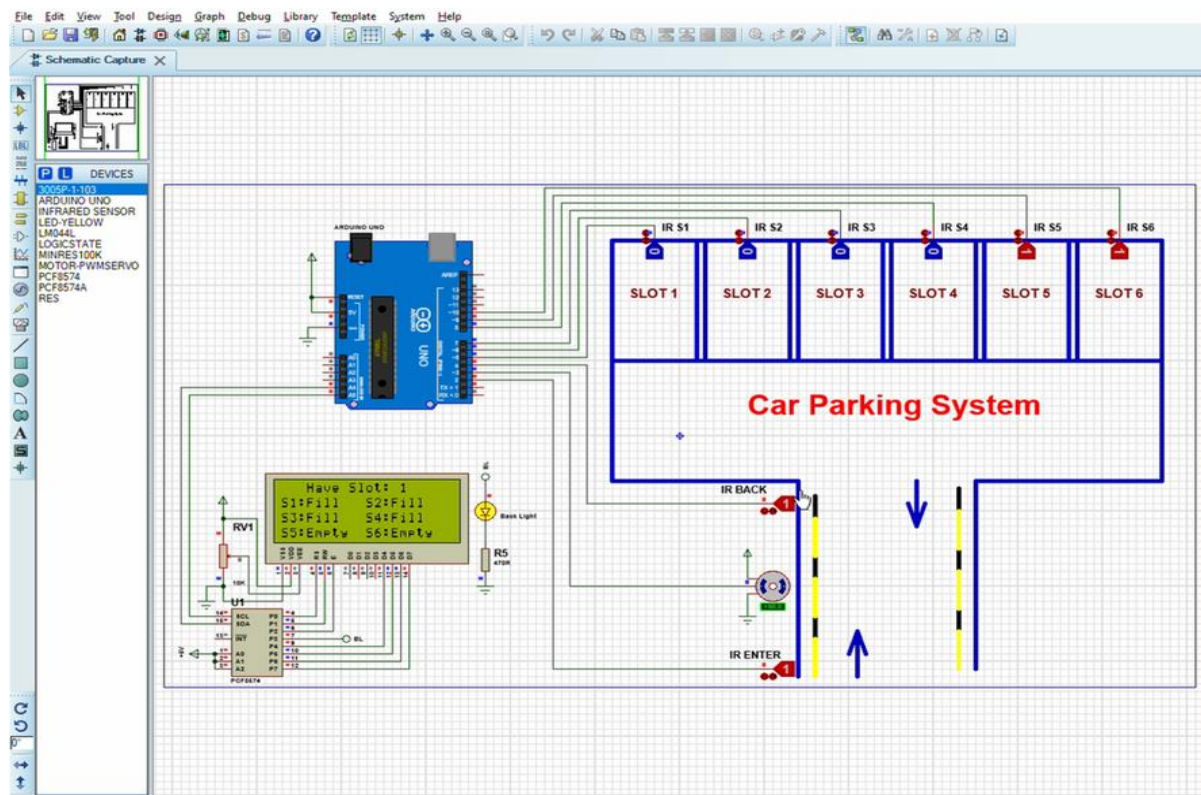


Figure 33: Implementation 2

Ai Face recognition

Aim of the system:

This system is designed to recognize every person pass front of the camera of the main door of the park and return his name and email address from the database, and if this person is not registered in our system the system will return “unknow.”

Tools:

To build this project I used Visual Studio code and python language, html, CSS and libraries like (pretrained model built with deep learning with accuracy 99.3%), OpenCV, NumPy, Flask framework and firebase-admin.

Running steps:

- 1) The system will open the camera as a video and receive frames from it.
- 2) The system will make a face detection process to detect if this frame has a person's face or anything other and detect his location.
- 3) If this frame has a person face system will make a face recognition process, if this person has an account on our system and already reserved in this park this process is implemented by extracting features from this face and compare it with features in the database and determine the distance between features (difference) and minimum distance expression of resemblance between two faces and it's mean this is the same person and system will return his name and email address from the database and return “unknown” if not.

- 4) display the output on html page connected with the system by using flask framework.
- 5) The system repeat step 2, 3 and 4 every second as well as the server is running to be able to recognize any person at any time.

This system is connected with firebase website to make all data in real time state.

This is screen form the output:

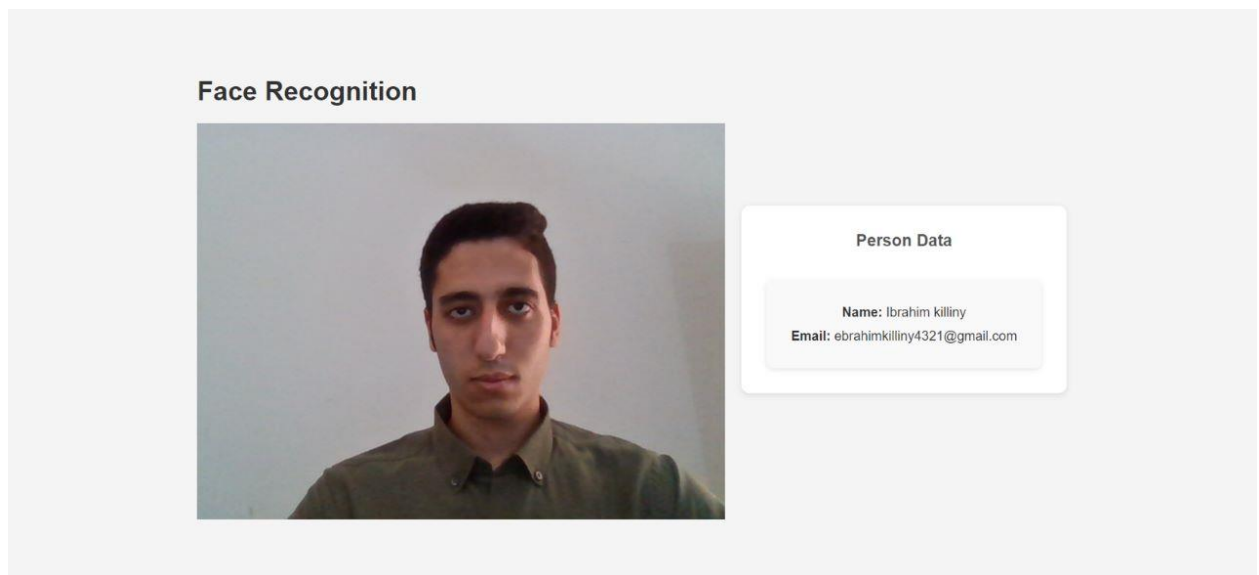


Figure 34: Output Face recognition

Output of unknown person

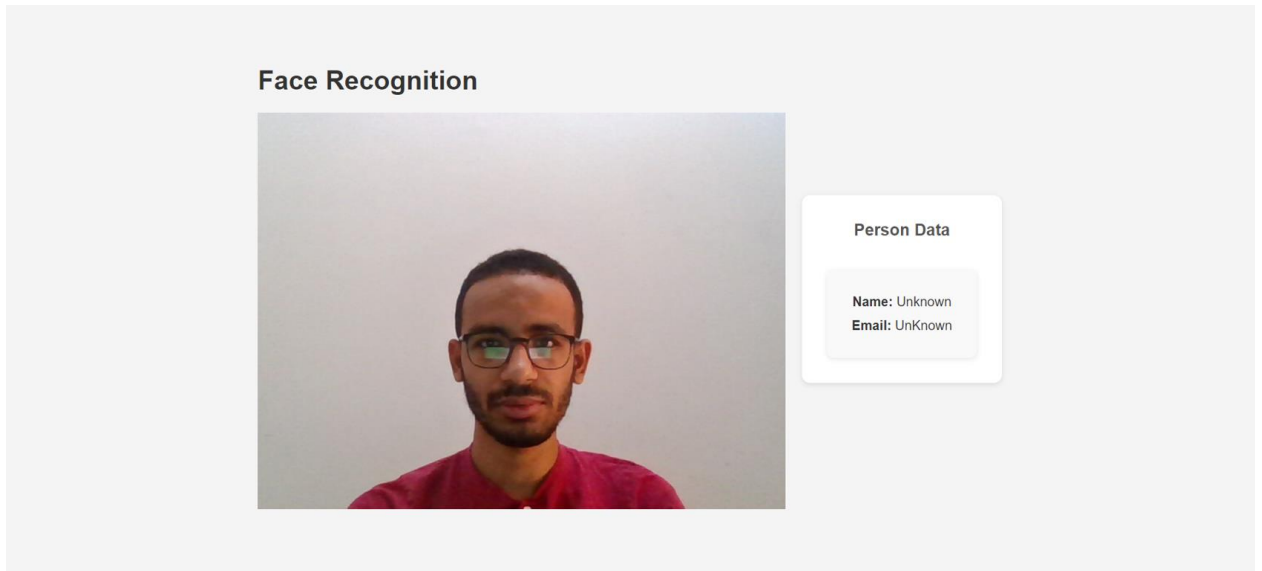


Figure 35: Output of unknown person

Chapter (5) Simulation

Welcome to the chapter on simulating our project! In this chapter, we will explore the process of simulating our project to test and evaluate its functionality, performance, and behavior. Simulation allows us to assess the project's capabilities, identify potential issues, and make informed decisions before the actual implementation.

Importance of Simulation

In this section, we will discuss the significance of simulation in the development lifecycle. We will explore the benefits of simulating our project, such as reducing development costs, mitigating risks, and optimizing performance. Simulation provides a controlled environment where we can observe and analyze how our project behaves under different scenarios and input conditions.

Simulation Techniques

We will explore simulation techniques that can be employed in our project.

Simulating Project Components

We will focus on simulating specific components of our project. We will break down the project into its constituent parts and discuss how to simulate each component individually. This may include simulating user interactions, system modules, data flow, or any other relevant aspects of our project. We will provide examples and guidelines to effectively simulate different components, ensuring a comprehensive evaluation of the project's behavior.

By simulating our project, we gain valuable insights into its behavior and performance, enabling us to make informed decisions and optimizations. Through this chapter, we will provide practical guidance on simulation techniques, component-level simulations, result analysis, and incorporating simulation findings into our project. Embracing simulation as an integral part of our development process sets the stage for a successful and well-optimized project. Let's dive into the world of project simulation

Simulation developed using React and Laravel, aimed at revolutionizing the parking process. This innovative application replicates the seamless experience of entering and exiting a parking area after making a reservation through a mobile app. By leveraging the power of React and Laravel, the simulation provides a realistic and interactive environment, allowing users to virtually navigate the parking process. From reserving a parking spot via the mobile app to arriving at the parking area, this simulation provides an immersive experience that mimics the actual flow of events. By using this simulation, users can familiarize themselves with the parking procedures, gain confidence in the reservation system, and optimize their parking experience. Say goodbye to uncertainty and embrace the convenience of this React and Laravel-powered simulation for a seamless parking process.

Simulate parking process in our system

First steps to enter parking

1- Start to make reservation via mobile app

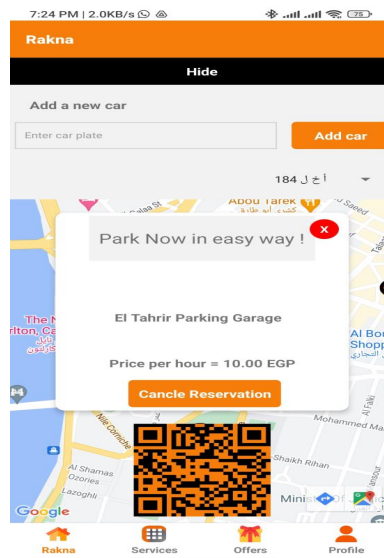


Figure 36: Reservation QR code

2- Then scan QR code when arrive to the parking area

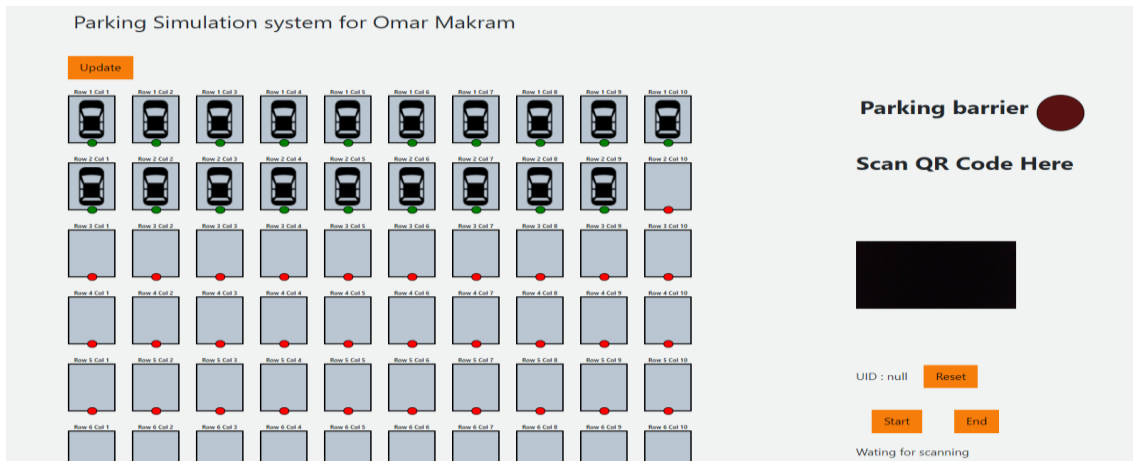


Figure 37: QR code Scanner

When scan QR code the code carry all information about reservation

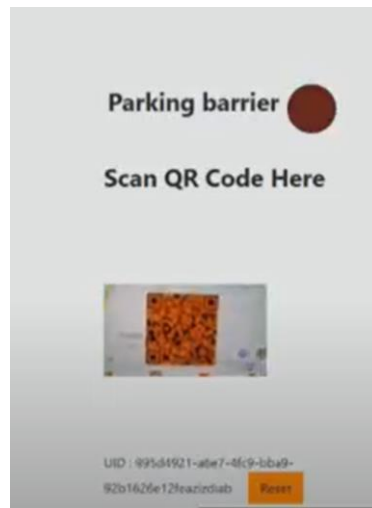


Figure 38: Get info from QR code

- 3- If reservation is valid the door open and appeared in screen go to row X : col Y and take image for person that enter with car



Figure 39: Parking Door

Door opened

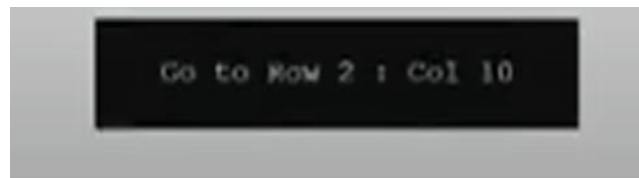


Figure 40: Parking Navigate

Screen to navigate client to go to certain area

4- Now car park in certain area and become in database not empty

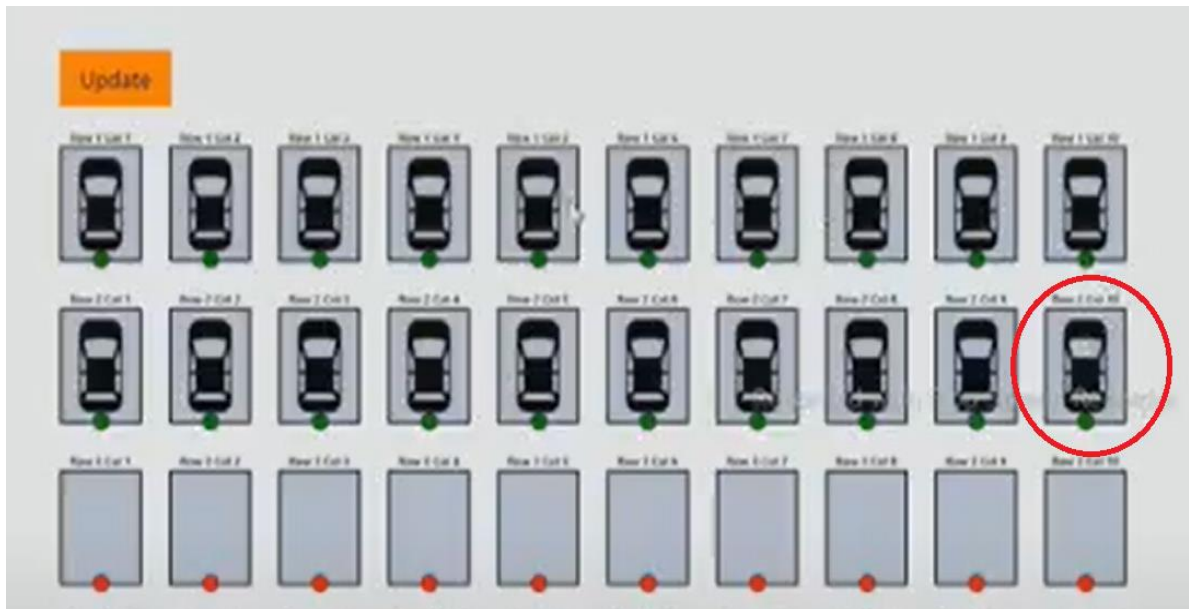


Figure 41: Simulate car park

Second steps to exit parking

1- Check face of person that try to exit

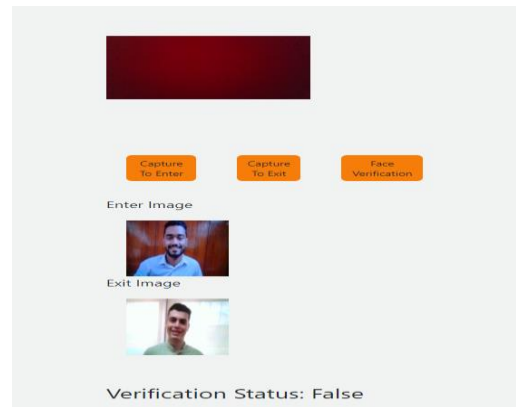


Figure 42: Face verification false

In this case face verification is false mean some try to exit with different car door not opened and alarm will fire

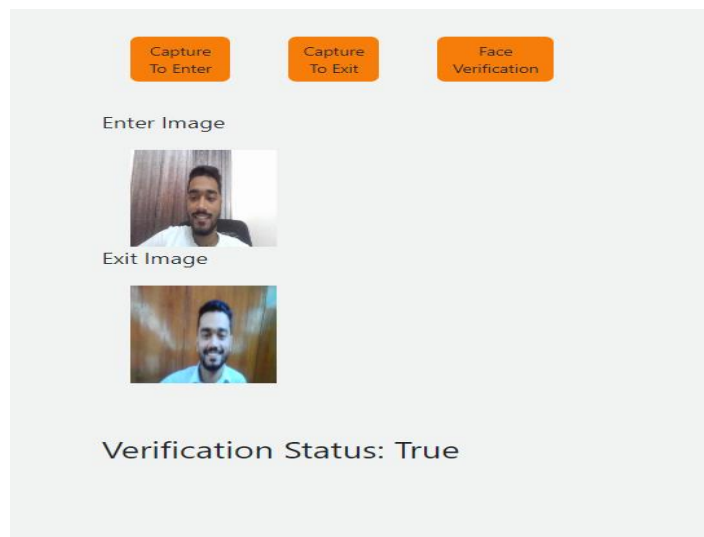


Figure 43: Face verification true

In this case face verification is true now can exit parking area

2- Scan QR code to exit

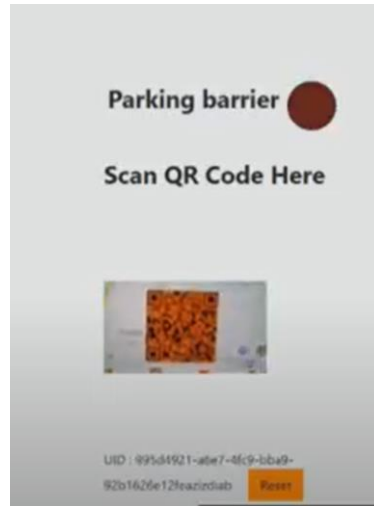


Figure 44: Scan QR code to exit

3- Door will open

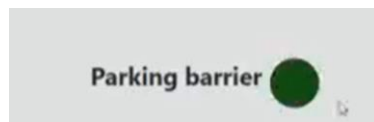


Figure 45: Parking door at exit position

4- The area become empty

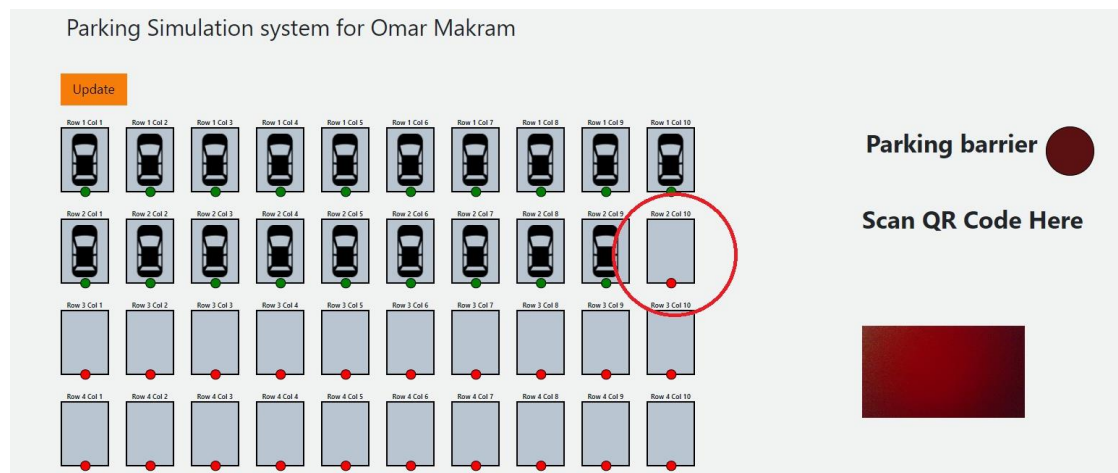


Figure 46: Simulate exit from parking

Chapter (6) Conclusion & future works

Conclusion

The project discusses several problems related to parking, such as overcrowding and not knowing where the garages are. As a result, people go to parking in the street, which causes crowded streets. The application that we designed works to solve these problems by providing an application on the phone that shows you the nearest available garages with empty spaces and the cost of parking. And system for organizing garage operations works without the need for any human element. It works on securing cars and organizing parking spaces in a better way. The process is done in a very easy way. Using Google Maps, the user is directed to the location of the garage. The reservation is linked to a QR code that carries the reservation details with the parking number, cost and time. Completing the process in an easy, safe and effective way, and works to solve the problems of street crowded.

Future works

Future works for software

Integration with Electric Vehicle (EV) Charging Stations: Provide information on nearby EV charging stations and their availability. Integration with EV charging networks can cater to the growing number of electric vehicle owners.

AR/VR Integration: Explore the integration of augmented reality (AR) or virtual reality (VR) technology to enhance the user experience. Users can visualize parking spots, navigate through parking structures, or receive directions through AR/VR overlays.

Integration with Vehicle Navigation Systems: Enable compatibility with vehicle navigation systems, such as in-car GPS or infotainment systems. This integration would allow users to seamlessly transfer directions from their vehicle to the app and vice versa.

Integration with Smart City Initiatives: Collaborate with local governments or smart city initiatives to leverage data and infrastructure. This collaboration can

facilitate efficient parking management and contribute to overall urban mobility goals.

Voice Recognition and Virtual Assistants: Integrate voice recognition capabilities and virtual assistants like Siri, Google Assistant, or Alexa. Users can make parking reservations, check availability, or get assistance through voice commands.

As part of future work for your mobile parking app, consider integrating a mechanical multilayered car park. Collaborate with architects and engineers to design and construct the facility, and update your app to include features such as real-time parking availability, reservation systems, navigation assistance, and vehicle recognition. Emphasize safety measures, implement maintenance alerts, and continuously improve the user experience. This integration expands parking capacity and offers users an efficient and convenient parking solution.

Future works for hardware

We will making IOT Car parking system.

The embedded system component:

- Microcontroller
- Ir sensor
- LCD display
- Stepper motor
- Esp8266
- Led

References

- [1] <https://www.geeksforgeeks.org/introduction-to-android-development/>
- [2] https://www.tutorialspoint.com/android/android_overview.html
- [3] <https://reactnative.dev/docs/getting-started>
- [4] <https://www.emse.fr/~picard/cours/ai/chapter01.pdf>
- [5] https://www.tutorialspoint.com/embedded_systems/es_overview.htm
- [6] <https://www.geeksforgeeks.org/introduction-of-embedded-systems-set-1/>
- [7] <https://www.techtarget.com/iotagenda/definition/embedded-system>
- [8] <https://www.datacamp.com/courses/introduction-to-data>
- [9] <https://byjus.com/maths/introduction-to-data/>
- [10] <https://edu.gcfglobal.org/en/access/introduction-to-databases/1/>

List of Figures

Figure 1: Android types	8
Figure 2: Features of android	9
Figure 3: Android Architecture	10
Figure 4 : Basic structure of an embedded system	30
Figure 5: ERD	45
Figure 6: Schema	46
Figure 7: Business Model	47
Figure 8: Use case diagram	51
Figure 9: App Onboarding screen	77
Figure 10: App Get started screen	78
Figure 11: App Sign in screen	79
Figure 12: App sign up	80
Figure 13: App Home screen	81
Figure 14: App parking details pop up	82
Figure 15: APP Reservation screen	83
Figure 16: App Profile screen	84
Figure 17: App services screen	84
Figure 18: App offers screen	85
Figure 19: Web login	86
Figure 20: Web Registration	87
Figure 21: Web User Dashboard	88
Figure 22: Web Admin Dashboard	89
Figure 23: Arduino Architecture	93
Figure 24: Arduino Pin Diagram	94
Figure 25: Hardware initial phase design	95
Figure 26: Hardware second phase design	96
Figure 27 : Arduino Uno 1	97
Figure 28 : Arduino Uno 2	98
Figure 29: Arduino Uno 3	99
Figure 30: : Arduino Uno 4	100
Figure 31: Arduino Uno 5	101
Figure 32 : Hardware Implementation 1	102
Figure 33 : Implementation 2	103

Figure 34: Output Face recognition	105
Figure 35: Output of unknown person	106
Figure 36: Reservation QR code	108
Figure 37: QR code Scanner	109
Figure 38: Get info from QR code	109
Figure 39: Parking Door	110
Figure 40: Parking Navigate	110
Figure 41: Simulate car park	111
Figure 42: Face verification false	112
Figure 43: Face verification true	112
Figure 44: Scan QR code to exit	113
Figure 45: Parking door at exit position	113
Figure 46: Simulate exit from parking	114

List of Tables

Table 1: Admin login	52
Table 2: Add new admin	53
Table 3: View user data	53
Table 4: Block user	54
Table 5: View admin data	54
Table 6: Block admin	55
Table 7: View parking owner data	55
Table 8: Block parking owner	56
Table 9: Open admin dashboard	56
Table 10: Registration	57
Table 11: Login	58
Table 12: Update user data	59
Table 13: Delete his account	60
Table 14: Search for nearest parking	60
Table 15: show nearest parking	61
Table 16: Make reservation	61
Table 17: Make monthly or weekly reservation	62
Table 18: Scan QR code to enter	62
Table 19: Scan QR code to exit	63
Table 20: Make payment	64
Table 21: Give rate	64
Table 22: Logout	65
Table 23: User authentication	66
Table 24: Generate QR code for reservation	67
Table 25: Calculate the cost of parking	68
Table 26: make payment	69
Table 27: Registration	70
Table 28: Login	71
Table 29: Update info	71
Table 30: Delete	72
Table 31: Open parking dashboard	72

Table 32: Logout.....	73
Table 33: Receive reservation request	73
Table 34: Confirm reservation	73
Table 35: Open baffle	74
Table 36: Close baffle	75
Table 37: Change value of parking area	75

At The End

Throughout this journey, we have come to realize that success is not an individual endeavor but a collective effort. We would like to express our deepest gratitude to our incredible team members, whose unwavering dedication and collaboration made this project possible. Their expertise, enthusiasm, and willingness to go above and beyond have been truly inspiring. We are immensely grateful for the invaluable mentorship and guidance provided by our esteemed professors. Their knowledge, insights, and continuous support have shaped our project and enriched our learning experience. We would also like to extend our heartfelt thanks to our families, whose unwavering support, understanding, and encouragement have been the pillars of strength throughout this challenging but rewarding journey. Their belief in our abilities and sacrifices made on our behalf have been immeasurable. Together, we have overcome obstacles, celebrated milestones, and grown both as individuals and as a team. This achievement would not have been possible without the collective efforts, unwavering support, and shared vision of our team members, professors, and families. We are truly grateful for the privilege of working with such remarkable individuals and for the lasting impact they have had on our personal and professional growth.

الْحَمْدُ لِلَّهِ الَّذِي هَدَانَا لِهَذَا وَمَا
كُنَّا لِنَهْتَدِيَ لَوْلَا أَنْ هَدَانَا اللَّهُ